MACHINE LEARNING
(CORE CONCEPTS)

:== PART-I ==:

(FOUNDATION)

→ **Machine Learning**

A subset of AI (Artificial Intelligence) and a process of finding mathematical function that maps inputs to outputs by learning patterns from data.

In simple words ~ Science of teaching computers to learn from data, similar to how human learn from experience.

ex- pridicting house prices, weather, email spam Filtering, Boimetric Authentication, Autonomous vehicles, Dynamic pricing etc.

→ **Traditional Programming Vs Machine learning**

• Give rules + data ↓ Get Results	• Give data + result ↓ Get Result (Model)
• Manual logic writing	• Learn patterns automatically

→ Real-life Machine learning Applications

- Youtube Recommendations:

  System learns our watch history to automatically recommend similiar content.

- Gmail Spam Detection:

  Gmail uses the massive collection of human actions as data to learn the difference between normal & spam emails, fully automating the filtering process.

- Self-Driving Cars/Autonomous Vechicles:

  Vehicles are fed massive amounts of camera data to learn how to identify stop signs, pedestrians & road boundaries.

- Face Unlock:

  Device scans & collect data points of our face from multiple angles to learn our features & securely unlock our device.

→ AI Vs ML Vs Deep learning (DL)

	AI	ML	DL
Definition	Macking Machine mimic human Intelligence	Machine learn patterns from data	Machines learn using deep Neural Network
Relation-ship	Parent field	Subset of AI	Subset of ML
Main Goal	Creat intelligent systems	Enable systems to learn from data	solve highly complex problems automatically
Need Data?	Not always	Yes	Yes, usually huge amounts
Learning Methods	Rules, logic, search, reasoning or learning	statistical learning from data	Multi-layer Neural networks
Feature Engineering	Manual	Mostly Manual	Automatic
Neural Network Required?	No	optional	Yes (essential)
Works with small Data?	Yes	Yes	Usually No
Computational Requirement	Low-Medium	Medium	very high (GPU/TPU)
Training Time	Fast	Moderate	slow
Accuracy on complex tasks	Moderate	High	Very high
Best For	Expert systems, robotics, AI games	Predictions, classification, recommendation	Image, speech, NL generative AI
Examples	chess Bot, self driving car	Loan prediction, spam detection, Recommendation systems	chatGPT, Facial Recognition, speech Recognition

→ Types of Machine Learning

(1) Supervised Learning:

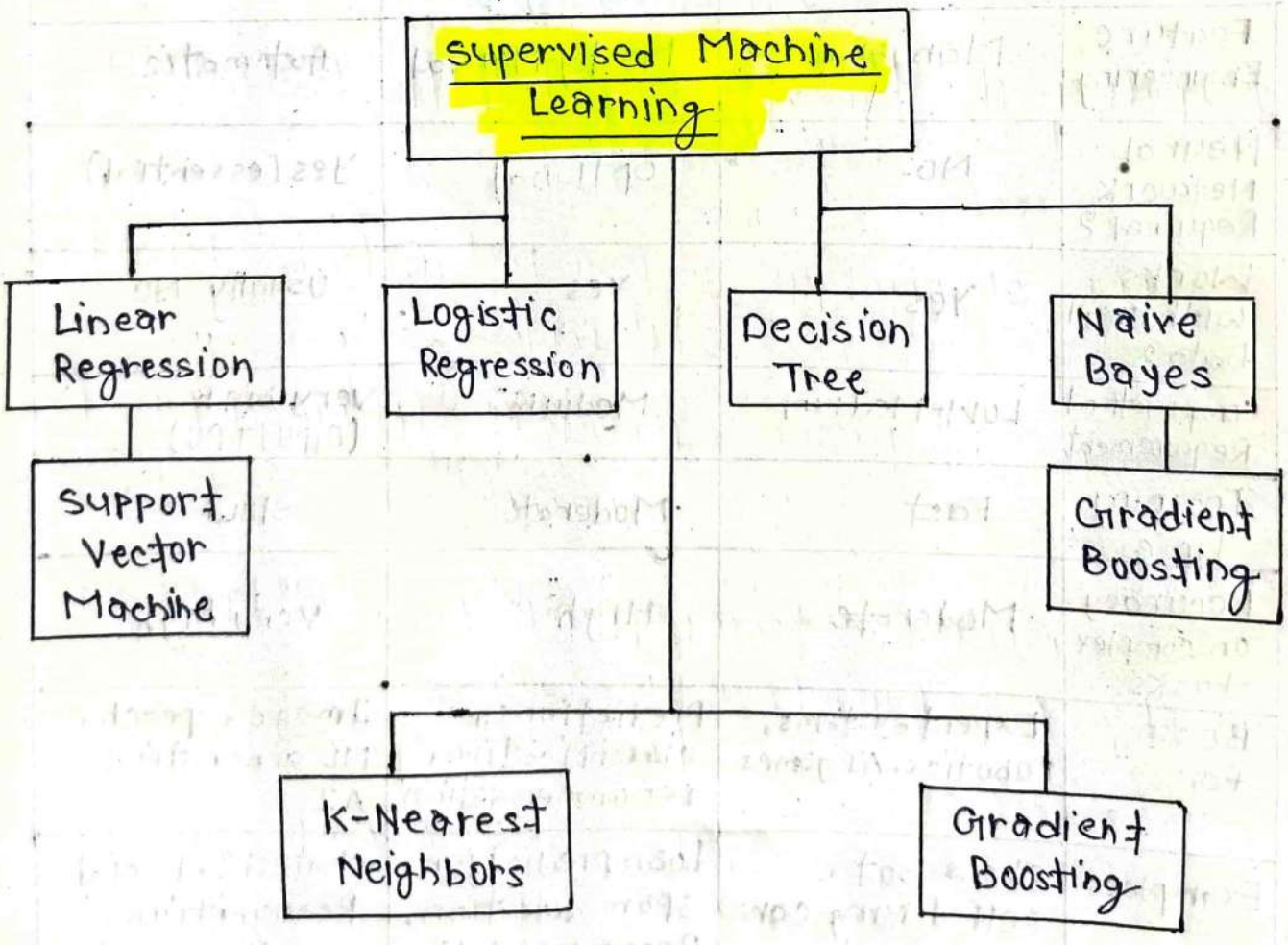
The Machine is fed both input variables and a correct output variable (the target), & 80-90% used in ML projects.

or

Model learns from labeled data, meaning every input (x) has a corresponding correct output (y).

ex- Email spam Detection, loan approval Prediction, Facial Recognition, Image classification etc.

↳ supervised learning Algorithms



(2) Unsupervised Learning

The Machine is given only input data, with no target outputs to predict.

It acts as a detective to find hidden pattern, groups similiar items (clustering) & reduce data complexity.

Ex- If we search "virat kohli" on Google, the system automatically groups thousands of different new stories into one specific cluster related to him.

(3) Reinforcement Learning: "Learning by Trail & error".

Mean Learning in which an agent learns by interacting with an enviroment & receiving rewards or penalties for its action.

Ex- An AI learning to play "Flappy Bird" game by repeatedly dying (penalty) until it figures out how to navigate the obstacles (rewards).

→ 11 steps for Building a Machine Learning Model

(1) Problem Definition - Define what you want to predict

(2) Data collection - Gather the required raw data.

(3) Exploratory Data Analysis -(EDA)- Understand the data, find Pattern, & detect outliers.

(4) Data preprocessing / cleaning - Handle missing value, duplicates & incorrect data.

(5) Feature Engineering - create new useful feature.

(6) Feature selection - Remove irrelevant or noisy features.

(7) Train-Test split - Divide data into training & testing sets.

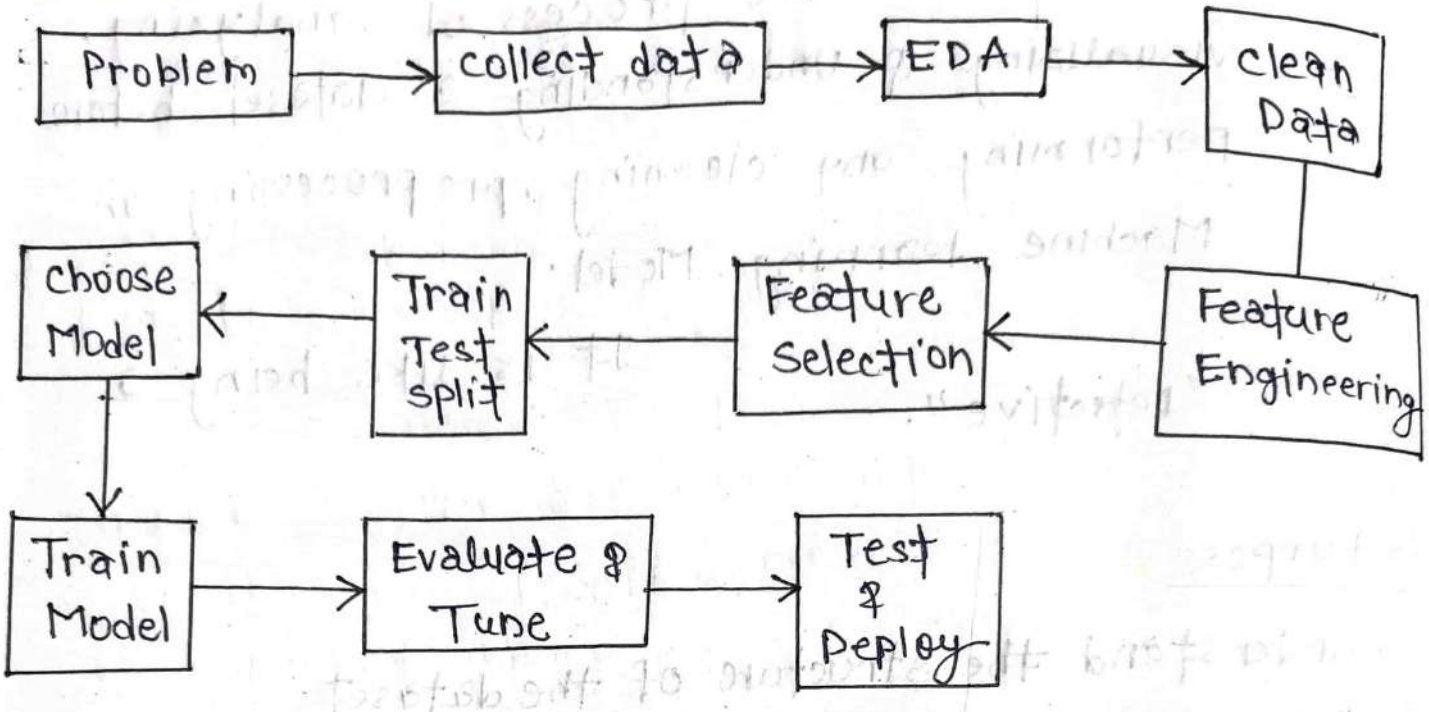
(8) Model selection - choose an appropriate ML algorithm.

(9) Model Training - Train the Model on the training data.

(10) Model Evaluation & Hyperparameter Tuning - Measure performance & optimize the Model.

(11) Model Testing / validation - Perform final testing before deployment.

→ ML Model Pipeline



→ EDA (Exploratory Data Analysis):

The process of analyzing, visualizing & understanding a dataset before performing any cleaning, pre-processing or Machine learning Model.

It is like being a "detective".

↳ Purpose

- Understand the structure of the dataset.
- Discover patterns & Relationship.
- Identify errors & inconsistencies.
- Detect Missing values & outliers.
- Understand how features affect the target variable.
- Generate hypotheses & insights.

↳ standard steps in EDA

(1) Viewing the Data

- head(), tail(), shape, info()
- for what column do we have in dataset & what type of data

(2) Summary statistics

- Finding Mean, Median, Mode, standard deviation, min & max of our data to understand its spread

(3) Value counts

- checking How many unique values exist in categorical columns.

(4) Missing Value Analysis

- Finding gaps, such as completly missing rows or Null values.

(5) Visualizations

- Histograms → Distribution of values
- Boxplots → outliers & spread
- Barplots → comparison of categories
- Correlation Heatmaps → linear relationships Between Numerical features
- scatter plots → Bivariate Relationships

(6) Target Variable Exploration

- Analyzing how the output we want to predict (ex- "charges") relates to the input variable.

→ Data cleaning

The process of Identifying & fixing errors, inconsistencies, Missing values, Duplicates & incorrect data to make the Dataset accurate and reliable for Machine learning.

"EDA" tells what is wrong in data, data cleaning fixes.

We spend 80% of our time cleaning data just to make the Math work.

↳ Key steps in Data cleaning

(1) Handling Missing Values (Nulls):

- checking which columns have Missing values (Nulls)
- strategies to handle -
  - * Drop Missing rows/columns (only if very few)
  - * Impute with :-
    - Mean / Median → for Numerical data
    - Mode → for categorical data
    - Advanced → Linear regression, KNN or interpolation (for future learning)

(2) Remove Duplicates

- Detect & drop exact duplicate rows

(3) Fix Data Types

- convert wrong types
  (ex- number stored as strings, dates as text)

(4) Handle Inconsistent Categories

- clean up categorical values like:
  - "Male", "male", "MALE" → should all become "male"
  - "yes", "yes", "y" → unify to one format

(5) Detect and Handle Outliers

- Use Boxplots, IQR or z-score
- Handle by:
  - Removing (if clearly wrong)
  - capping (ex- to 95th percentile)

(6) Fix logic or Domain Errors

- ex- age = -5 is invalid, or BMI = 150 likely an error
- can replace with mean, median or remove

→ Data Preprocessing :

It is about Transforming valid, cleaned data into a usable, numerical format that machine learning models can actually process & learn from.

↳ (1) Encoding Categorical Variable

Converting categorical (text) data into numerical form Because machine learning algorithms work with numbers, not words.

- **Label Encoding** : Used for simple or ordered categories.
  ex- convert "Male" into 0 & "Female" into 1 in a dataset.

- **One-Hot Encoding** : Used for unordered categories.
  ex- Regions - North, South, East West). It create a brand new column for each region, filling them with 1 (if true) & 0 (if false).

(2) Feature Transformation:

Process of changing the distribution or Mathematical form of features to make data more suitable for ML algorithms

Mainly used for-

- Left skewed data - Most values are LOW, few are very High (ex - 30k, 35k, 40k, 45k, 50k, 10 lakh)
- Right skewed data - Most value are High, few very low.
- Non-Normal distributions (ex - 20, 85, 88, 90, 92, 95, 9)
- Reducing extreme values

(3) Feature Scaling (Normalization or standardization)

Process of Bringing numerical numerical features with different ranges to a similar scale so that machine learning algorithms treat all features fairly.

* Normalization (Min & Max scaling): scales values Between 0 and 1.

* standardization (z-score scaling): Transforms data to have mean 0 & std 1.

→ Feature Engineering

Creating new and useful features (columns) from transforming existing data to expose useful patterns that ML Models can learn from.

↳ Common Feature Enginnering Techniques in ML

(1) Target Based Flags: These are binary (0/1) features created from existing feature to capture patterns related to the target variable.

(2) Mathematical combinations

Multiplying or Dividing two existing columns to create a new, highly predictive score.

(3) Time-Based Features: splitting a raw "Data" column into three new seperate columns: "Day", "Month" & "year".

(4) Binning: Grouping continuous numbers into categories.

ex- taking exact ages & putting them into "10-20", "20-30" age group bins.

→ Feature selection :

Process of choosing important features and removing irrelevant or unnessary features from the dataset to improve model performance

ex- 36 columns after feature engineering & encoding, using all of them migh confuse. the model & put extra stress on model so we select only useful columns.

↳ Importance

- Improve accuracy
- Reduces overfitting
- Removes irrelevant Inoisy features
- Makes training faster
- Reduces memory & computation requirements.
- Improve model interpretability

↳ Feature selection Methods

(1) Filter Methods (Pure statistics) : Using tests to mathematically prove which input has the strongest relationship with target output

- correlation Matrix → Remove highly correlated features.
- chi-square test (categorical vs categorical)
- ANOVA F-Test (numerical vs categorical target)

(2) Embedded Methods : Relying on built in ML algorithms that automatically shrink useless features to zero & score the importance of good features.

- Lasso Regression → shrinks coefficients to 0
- Tree-Based Models (Random Forest, XGBoost)
  ↳ Feature importance scores.

→ Decision Tree is a Model that makes predictions by asking a serious yes or no question

Random Forest is a collection of Many Decision Trees that work together to make better predictions

PROJECT : 1
INSURANCE COST PAIDICTION PROJECT

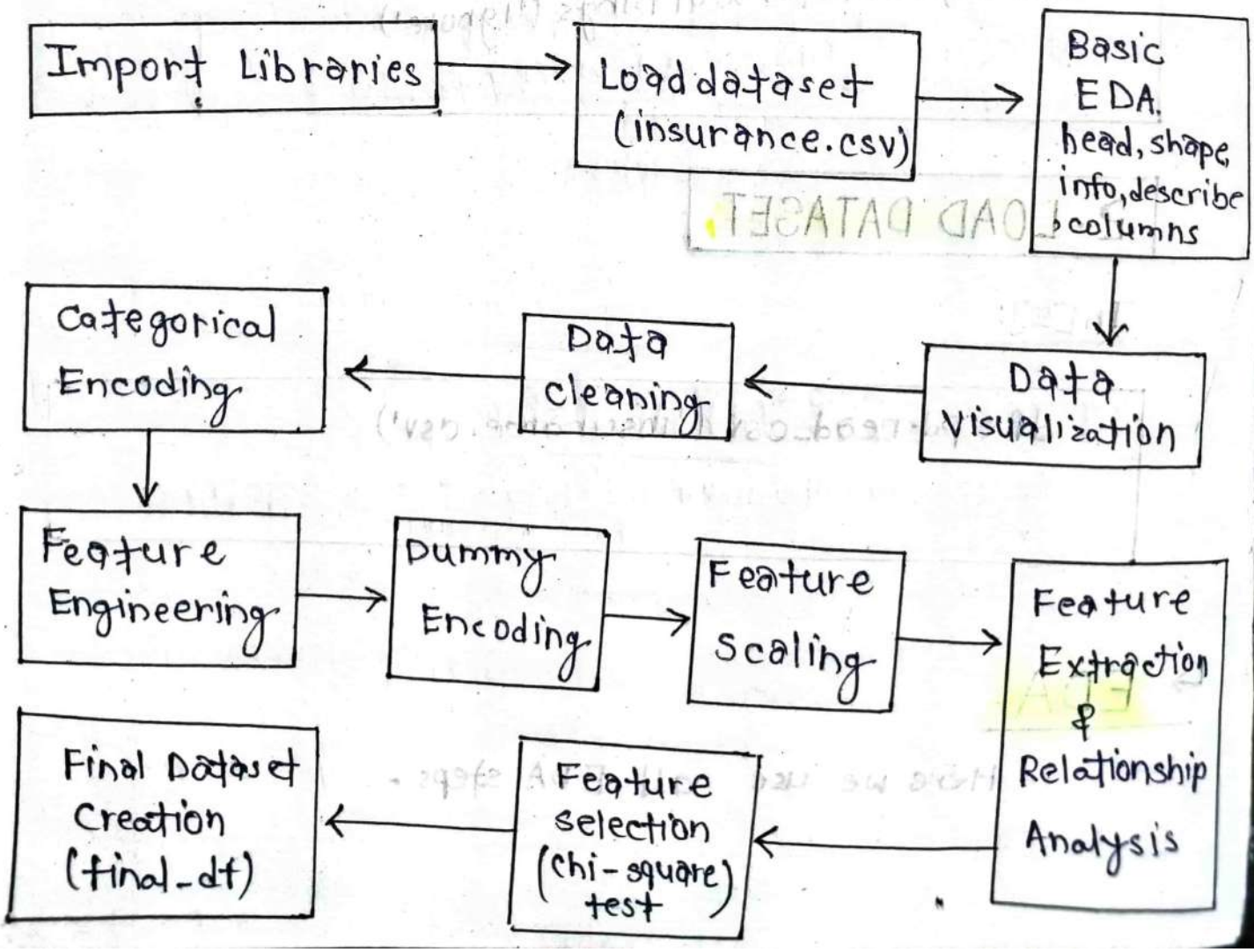
↳ overview

In this Project w' goal is to predict a person's Medical insurance charges based on their information.

But here we are not building full Model we will go to Feature engineering & Extraction. we build full fleged ML Model in further parts.

file - insurance.csv

↳ Pipeline of our project



1. IMPORT LIBRARIES

In[1]:

```
import numpy as np
# for numerical calculation & arrays

import pandas as pd
# loading & Manipulating data

import seaborn as sns
# Statistical visualization

import matplotlib.pyplot as plt
# for plotting graphs

import warnings
# for handling warning Messages

warnings.filterwarnings('ignore')
# for hide unnessary warnings.
```

2. LOAD DATASET

In[2]:

```
df = pd.read_csv('insurance.csv')
# to load the insurance dataset into DataFrame
```

↳ EDA

Here we use all EDA steps.

3. BASIC EDA

In[3]:

```
df      # for display Dataset

df.shape
# for getting no. of rows & columns.

df.head()
# for Display the first 5 rows

df.info()
# for Display data type & Non null values

df.describe()
# for getting the statistical summary of
  numerical columns like std, min, max, mean etc.

df.isnull().sum()
# used to count missing values
```

4. DISTRIBUTION ANALYSIS

In [4]:

```
numeric_columns = ['age', 'bmi', 'children', 'charges']
 # Used to store numeric columns
for col in numeric_columns:
 plt.figure(figsize=(6,4))
 # Used to create a figure
 sns.histplot(df[col], kde=True, bins=20)
 # for histrogram, Total bins = 20 and kde=True
   for bell curve
```

5. COUNT PLOTS

In [5]:

```
sns.countplot(x=df['children'])
#for count each children category & graph
sns.countplot(x=df['sex'])
#for counting Males & female in graph
sns.countplot(x=df['smoker'])
#Used to count smokers & Non smoker in graph
```

6. OUTLIER DETECTION

In [6]:

```
for col in numeric_column:
    plt.figure(figsize=(6,4))
    #Used to create a figure with 600x400 pixels.
    sns.boxplot(x=df[col])
    # for making box plot & Used to Detect
      Outliers in numerical column
```

7. CORRELATION HEATMAP

In [7]:

```
plt.figure(figsize=(8,6))
sns.heatmap(df.corr(numeric_only=True),
annot=True)
#for visualize correlation between numerical
 feature & creates Heatmap, annot for co-relation no.
```

Data cleaning & Preprocessing

8. CREATE COPY OF DATASET

In [8]:

```
df_cleaned = df.copy()
# for creating copy of original dataset.
df_cleaned.head()
#for displaying first 5rows
df_cleaned.shape
```

9. REMOVE DUPLICATES

In [9]:

```
df_cleaned.drop_duplicates(inplace=True)
# for removing Duplicates rows permanently
  inplace=True → Change in existing column only drop
df_cleaned.shape
```

10. MISSING VALUES & DATA TYPES

In [10]:

```
df_cleaned.isnull().sum()
# for counting Missing value(Null value)
df_cleaned.dtypes
# for Display Data types of all columns
```

inplace = True → "Modify the original dataframe"

inplace = False → "create & return a Modified Dataframe copy"

11. ENCODING SEX

In [11]:

```
df_cleaned['sex'].value_counts()
#used to count unique value in 'sex' column
df_cleaned['sex'] = df_cleaned['sex'].map
                    ({"male":0, "female":1})
#for converting Male & female into numerical
  value 0 & 1
df_cleaned.head()
#for displaying updated dataset
```

12. ENCODING SMOKER

In [12]:

```
df_cleaned['smoker'].value_counts()
# Used to count unique no. in 'smoker' column
df_cleaned['smoker'] = df_cleaned['smoker'].
        map({"no":0, "yes":1})
#Used to convert smoker categories into
  numerical values.
df_cleaned
```

13. RENAME COLUMNS

In [13]:

```
df_cleaned.rename(columns={'sex':'is_female',
        'smoker':'is_smoker'}, inplace=True)
#renaming columns 'sex' to 'is_female & 'smoker' to
                                    'is_smoker'
df.cleaned.head()
```

14. REGION ENCODING

In[14]:

```
df['region'].value_counts()
# for count records in each region
df_cleaned = pd.get_dummies(df_cleaned ,
columns = ['region'] ,drop_first = True)
# Used to convert region categories into
  dummy variables , drop_first = True - mean
  drops the first category column.
df_cleaned.head()
```

15. DATA TYPE CONVERSION

In[15]:

```
df_cleaned = df_cleaned.astype(int)
# to convert all columns into integer data type
df_cleaned
```

↳ Feature Engineering & Extraction

16. BMI DISTRIBUTION

In[16]:

```
sns.histplot(df['bmi'])
# to visualize the distribution of BMI values
  in Histogram
```

17. BMI CATEGORIZATION

In [17]:

```
df_cleaned['bmi_category'] = pd.cut(df_cleaned
    ['bmi'], bins=[0, 18.5, 24.9, 29.9, float('inf')],
    labels=['Underweight', 'Normal', 'overweight',
    'obese'])
#for convert continuous BMI value into categories
  Mean new category 'bmi_category' with 'Underweight',
  'Normal', 'overweight', 'obese' values
df_cleaned
```

18. BMI DUMMY ENCODING

In [18]:

```
df_cleaned = pd.get_dummies(df_cleaned,
columns = ['bmi_category'], drop_first = True)
# used to convert BMI category into dummy
  variables

df_cleaned = df_cleaned.astype(int)
# for convert colums into integer datatype.

df_cleaned.head()

df_cleaned.columns
# display all column names.
```

19. FEATURE SCALING

In [19]:

```
from sklearn.preprocessing import standardScaler
# import standard scaler, standard scaler is used
  to resize our data so that it has a mean of
  0 & a standard deviation of 1
# sklearn.preprocessing → is a toolkit to transform
  raw data into a clean format that ML Models
  can Understand.

cols = ['age', 'bmi', 'children'] # store numerical columns
                                    for scaling
scaler = standardScaler()
# for create la scaler object

df_cleaned[cols] = scaler.fit_transform(df_cleaned
                                        [cols])
# used to standardize numerical features.

df_cleaned.head()
```

20. PEARSON CORRELATION ANALYSIS

In [20]:

```
from scipy.stats import pearsonr
# to calculate pearson correlation coefficients.
selected_features = ['age', 'bmi', 'children', 'is_female',
'is_smoker', 'region_northwest', 'region_southeast',
'region_southwest', 'bmi_category_Normal', 'bmi_category_
 overweight', 'bmi_category_obese']
# List of features to check against target &
  Used to store feature to compare with charges
```

```
correlations = { features: pearsonr(df_cleaned[feature],
df_cleaned['charges'])[0]
for features in selected_features }
# for converting correlation result into a DataFrame.
correlation_df = pd.DataFrame(list(correlations.
items()), columns = ['Feature', 'Pearson Correlation'])
#Used to convert co-relation into DF
correlation_df.sort_values(by='Pearson Correlation',
ascending = False)
# Used to sort Features from highest to lowest
correlation.
```

21. CHI-SQUARE FEATURE SELECTION

In [21]

```
cat_features = ['is_female', 'is_smoker', 'region_northwest',
                'region_southeast','region_southwest',
                'bmi_category_Normal','bmi_category_Overweight',
                'bmi_category_obese']
#for storing Categorical Features
from scipy.stats import chi2_contingency #chi-square test
alpha = 0.05
df_cleaned['charges_bin'] = pd.qcut(df_cleaned['charges'],
                                    q=4, labels=False)
# for divide charges into four equal groups
chi2_result = {}
# storing chi-square test
```

In[22]

```
for col in cat_features:
    contingency = pd.crosstab(df_cleaned[col],
                              df_cleaned['charges_bin'])
    # pd.crosstab → used to create contingency
      Table
    chi2_stats, p_val, _, _ = chi2_contingency
                              (contingency)
    # for calculate thi chi-square statistic
      and p-value
    decision = 'Reject Null (Keep Feature)' if
    P_val < alpha else 'Accept Null (Drop feature)'
    # for deciding whether the feature is
      significantly related to charges.
    chi2_results[col] = {'chi2_statistic': chi2_stat,
                         'P_value': p_val,
                         'Decision': decision}.
      # to store the test result for the current
      feature
chi2_df = pd.DataFrame(chi2_results).T
# convert the results dictionary into a pdaFrame
chi2_df = chi2_df.sort_values(by='P_value')
# Used to sort features from most significant
  to least significant
chi2_df
# to Display the final chi-square result table
```

In[23]

```
final_df = df_cleaned [['age', 'is_female', 'bmi',
        'children', 'is_smoker', 'charges' ,
        'region_southeast', 'bmi_category_obese']]
# to create the final dataset with selected
  important features.
final_df
# Disploy final dataset.
```

* Pearson correlation Measures how strongly two numerical variable are related to each other

PROJECT 2:
HEART DISEASE PREDICTION

→ Project overview

The goal is to analyze patient health data & prepare it for a machine learning model that can predict whether a person has heart disease.

→ Dataset (Df) information

- Dataset file : Heart.csv
- Rows : 918 Patients
- columns : 12 features
- Target Variable : HeartDisease

→ EDA

1. IMPORT LIBRARIES

In[1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
# used to handle warning messages
warnings.filterwarnings('ignore')
# for hiding warnings
```

2. LOAD DATASET

In[2]:

```
df = pd.read_csv("heart.csv")
```

↳ EDA

3. BASIC EDA

* Here we are performing EDA & cleaning mean both

In[3]:

```
df.head()
df.columns
df.shape  # No. of rows & columns
df.info()  # displays data types & Non-null
df.describe()# for statistical summary
df.duplicated().sum() # count Duplicate rows
df.isnull().sum()  # count Null
```

4. TARGET VARIABLE ANALYSIS

In[4]:

```
df['HeartDisease'].value_counts().plot(kind="bar")
```

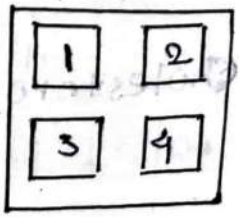
↳ # used to visualize the No. of patients with and without heart disease.

5. DISTRIBUTION ANALYSIS

In[5]:

```
def plotting(var,num):
# to create a histogram plotting function
    plt.subplot(2,2,num)
    # Defining subplot position
    sns.histplot(df[var],kde=True)
    # plot histogram with bell curve

plotting('Age',1)
# Visualize Age Distribution at subplot on position 1
plotting('RestingBP',2)
plotting('Cholesterol',3)
plotting('MaxHR',4)
plt.tight_layout()
# for adjust spacing between plots.
```

* subplots - It allows to create Multiple smaller plots side by side on a single blank canvas
ex- i's 4 subplots .(4, 4)



6. CHOLESTEROL ANALYSIS

In [6]

```
df['Cholesterol'].value_counts()
# for inspecting cholestrol value & identify
  abnormal values like 0.
```

* In histogram we can some errors in cholestrol & Resting BP Both are 0 but a human cannot have zero(0) cholestrol and 0 Resting BP.

So we take as error and replace the zero (0) with their mean values.

7. CHOLESTROL CLEANING

In [7]

```
Ch_mean = df.loc[df['Cholesterol'] != 0,
          'Cholesterol'].mean()
# for calculating mean cholesterol excluding
  zero values
df['Cholestrol'] = df['Cholesterol'].replace(0,
                   Ch_mean)
# Used to replace invalid cholesterol values
  with mean.
df['Cholesterol'] = df['Cholesterol'].round(2)
# to round cholesterol values to 2 decimal place
```

8. RESTING BP CLEANING

In[8]:

```
resting_bp_mean = df.loc[df['RestingBP'] != 0,
                         'RestingBP'].mean()
# Calculate Mean of RestingBP excluding zero values.

df['RestingBP'] = df['RestingBP'].replace(0,
                                          resting_bp_mean)
# replace invalid RestingBP values with mean

df['RestingBP'] = df['RestingBP'].round(2)
 # Round RestingBP values to 2 decimal places
```

9. DISTRIBUTION ANALYSIS AFTER CLEANING

In[9]:

```
def plotting(var, num):
    plt.subplot(2,2, num)
    sns.histplot(df[var], kde=True)
Plotting('Age', 1)
Plotting('RestingBP', 2)
Plotting('cholesterol', 3)
Plotting('MaxHR', 4)
plt.tight_layout()
```

10. CATEGORICAL FEATURE ANALYSIS

SEX VS HEART DISEASE

In [10]

```
sns.countplot(x = df['Sex'], hue= df['HeartDisease'])
# to compare heart disease case across gender &
  visualize
```

CHEST PAIN TYPE VS HEART DISEASE

In[11]

```
sns.countplot(x=df['ChestPainType'], hue=df
['HeartDisease'])
# compare heart disease cases across chest pain
  types.
```

FASTING BLOOD SUGAR VS HEART DISEASE

In[12]

```
sns.countplot(x=df['FastingBS'], hue= df[
                'HeartDisease'])
# compare heart disease cases across fasting
  blood sugar groups
```

11. BOX PLOT ANALYSIS

In [13]:

```
sns.boxplot(x = 'Heartdisease', y= Cholesterol',
            data = df)
# for detecting cholesterol spreads and outlier
  by heart disease class & visualize
```

12. VIOLIN PLOT ANALYSIS

In[14]:

```
sns.violinplot(x='HeartDisease', y='Age', data=df)
# Used to compare age distribution between
  Heart disease classes
```

13. CORRELATION ANALYSIS

In[15]:

```
sns.heatmap(df.corr(numeric_only=True), annot=True)
# to visualize correlation among numeric features.
```

↳ Data Pre-Processing & cleaning

14. ONE-HOT ENCODING

In[16]:

```
df_encode = pd.get_dummies(df, drop_first=True)
# to convert categorical columns into numerical
  column
df_encode # Display encode dataset
```

15. CONVERT BOOLEAN TO INTEGER

In[17]:

```
df_encode= df_encode.astype(int)
# to convert True/False value into 1/0
df_encode
```

16. FEATURE SCALING

In [18]:

```
from sklearn.preprocessing import StandardScaler
numerical_cols = ['Age', 'RestingBP', 'Cholesterol',
                  'MaxHR', 'Oldpeak']
# to store numerical column name
scaler = StandardScaler()
# create scaler object
df_encode[numerical_cols] = scaler.fit_transform
        (df_encode[numerical_columns])
# Used to standardize numerical Features
df_encode.head()
```

17. RANDOM FOREST FETURE

- We used this to find the features that contribute the most to heart disease prediction

In [19]:

```
from sklearn.ensemble import RandomForestClassifier
# Used to Import Random Forest
X = df_encode.drop('HeartDisease', axis=1)
# to store input features
y = df_encode['HeartDisease']
# store target variable.
rf = RandomForestClassifier(n_estimators=100,
     random_state=42
#Create Random Forest model
```

```
rf.fit(X,y)
# used to train model
rf_importance = pd.DataFrame({'Feature' : X.columns,
                'RF Importance' : rf.feature_importances_})
# create feature importance table.

rf_importance = rf_importance.sort_values(
        by = 'RF Importance', ascending = False)
# used to sort features from highest to lowest
  importance

rf_importance
```

18. MUTUAL INFORMATION

- Used to find the features most strongly related to Heart Disease.

In[20]:

```
from sklearn.feature_selection import
mutual_info_classif
# Import Mutual Information

mi_scores = mutual_info_classi(X,y,
            random_state = 42)
# calculate MI scores
mi_importance = pd.DataFrame({'Feature' : X_columns,
                'MI Score' : mi_scores
# Used to create MI score table.
```

```
mi_importance = mi_importance.sort_value(
                    by = 'MI score',
                    ascending = False')
# sort Features from highest to lowest score
mi_importance
# Display MI score
```

19. COMMON IMPORTANT FEATURE

- Used to select the most reliable features agreed upon by both methods.

In [21]:

```
rf_top = set(rf_importance['Feature'].head(10)
# to get top 10 Random Forest features.
mi_top = set(mi_importance['Feature'].head(10)
# to get top 10 Mutual Information Features
common_features = rf_top.intersection(mi_top)
# to find features importont in both methods

common_features
```

* We used "Random Forest Feature' & "Mutual Information" to identify important features, then selected "the common important features" to build a more accurate & reliable heart disease prediction model.

20. CREATE FINAL SELECTED DATASET

In [28]:

```
X_selected = df_encode[list(common_features)]
# Used to create dataset with selected features
  only

df_selected = X_selected.copy()
# to create a copy of selected features

df_selected['HeartDisease'] = y
# to add target column back

df_selected.head()
# Display first 5 rows of final dataset
```

- Used to create the final dataset containing only Important features.

PART-2

(SUPERVISED MACHINE LEARNING)

→ Some Important Note

- **code simplicity Vs Mathematical depth**: Creating a model like linear regression or logistic regression is highly straight forward in practice, requiring only six to seven lines of code.

  However, Understanding the Mathematical intuition, gradient descent & cost functions is what requires deeper ~~study~~ & multiple viewings.

- Focus on Understanding concepts such as:
  - Cost
  - Gradient Descent
  - Model optimization

→ Supervised Learning types

It is divided into two categories:

(1.) Regression Algorithm

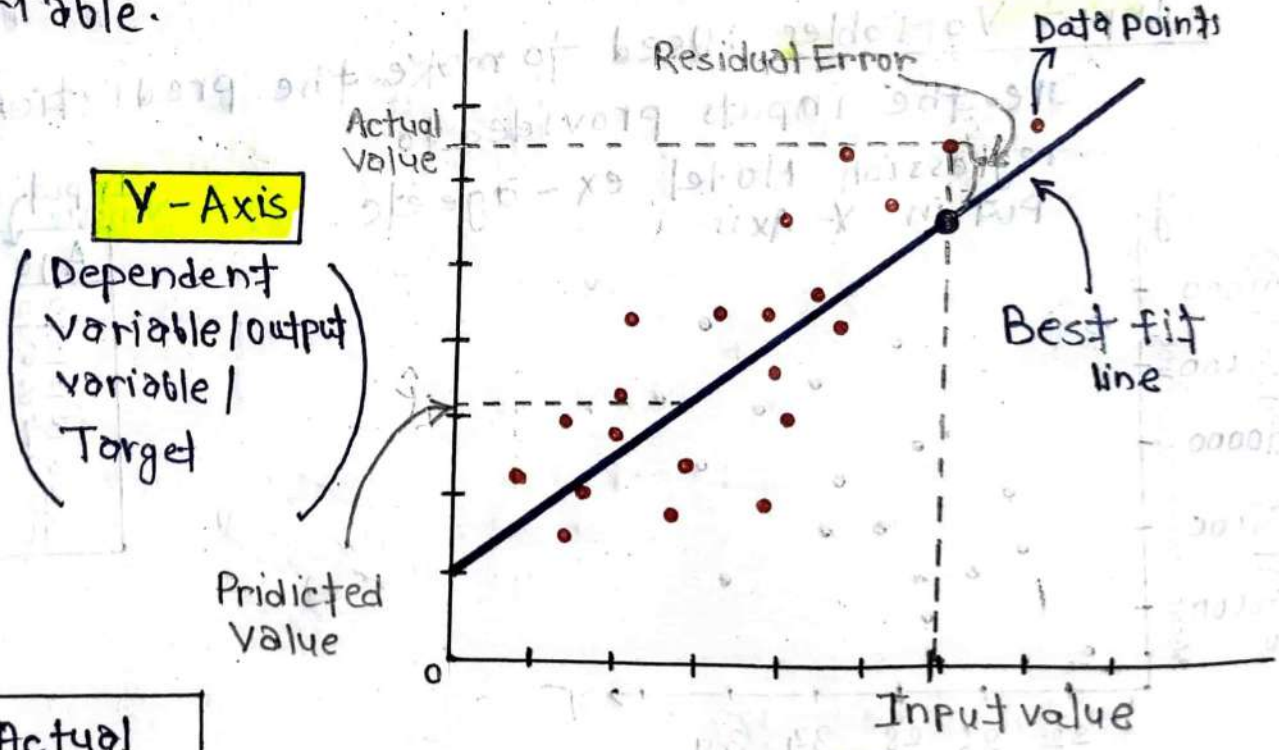
- Predicts numerical values.
- ex:
  - House Price Prediction
  - salary Prediction
  - Car price Prediction

(2) classification Algorithms

- Predicts categories or classes.
- Ex:
  - spam or Not spam
  - Disease Detection
  - Survived (1) or Not survived (0)

→ Regression Algorithm

In this Machine learning algorithm used to predicting a continuous numerical value by learning the relationship between input features and the target variable.



Y-Axis
(Dependent variable / output variable / Target)

X-Axis
(Independent variable / Input variable / Feature)

- **Datapoints**: Actual observed values from the dataset

↳ Continuous Vs Categorical Data

Continuous

(1) Predicts Numerical values

(2) Output can be any value within a range

(3) Ex: salary - ₹45000/-

Ex: House Price = ₹2500000/-

Categorical Data

(1) Predicts categories / classes

(2) output belongs to predefined classes

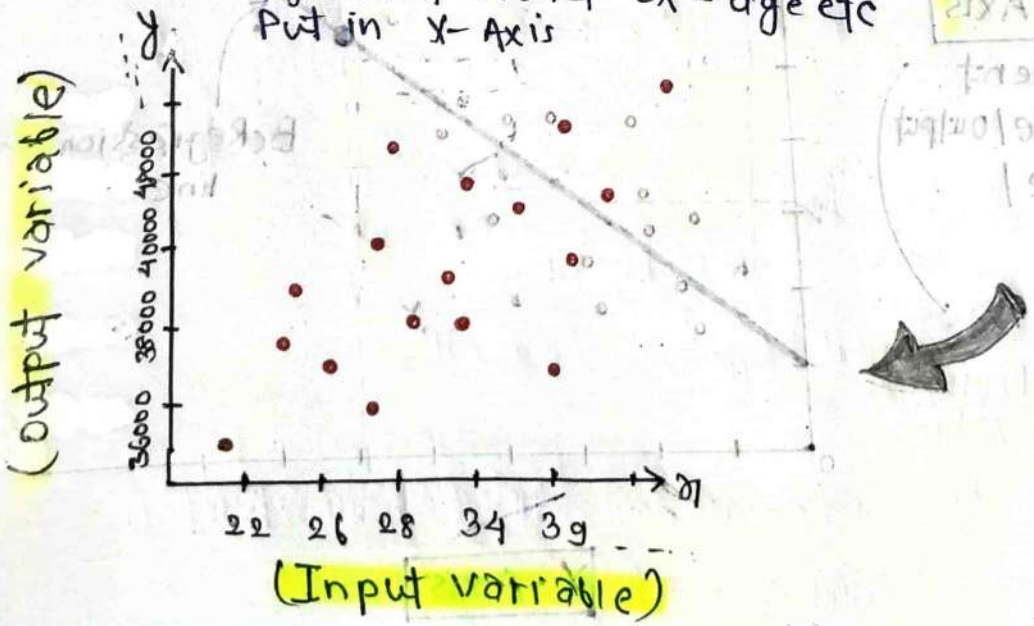
(3) Ex: Pass / Fail

Ex: Spam / Not spam

↳ Input & Output Variables

(X-Axis)

• **Input Variables** : Used to make the prediction. These are the inputs provided to regression Model ex - age etc Put in X-Axis

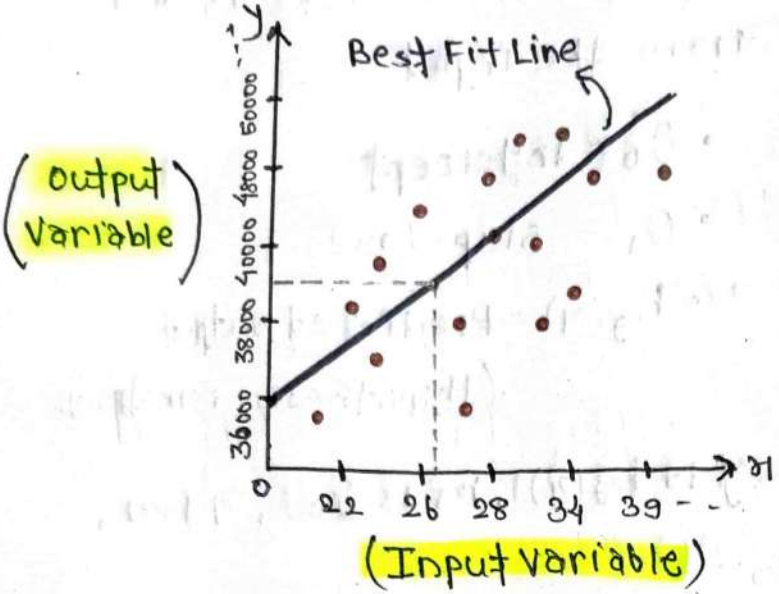


Input variable → Age	output variable → salary
22	36000
26	38000
28	40000
34	48000
39	52000
42	⋮
46	

• **Output Variable** (Y Axis) : A continuous numerical value that the Model aim to predict based on the input variables ex - salary etc.

↳ **Best-Fit Line**: In Linear Regression, a scatter plot is created with input variables on the x-Axis & the output on the y-Axis.

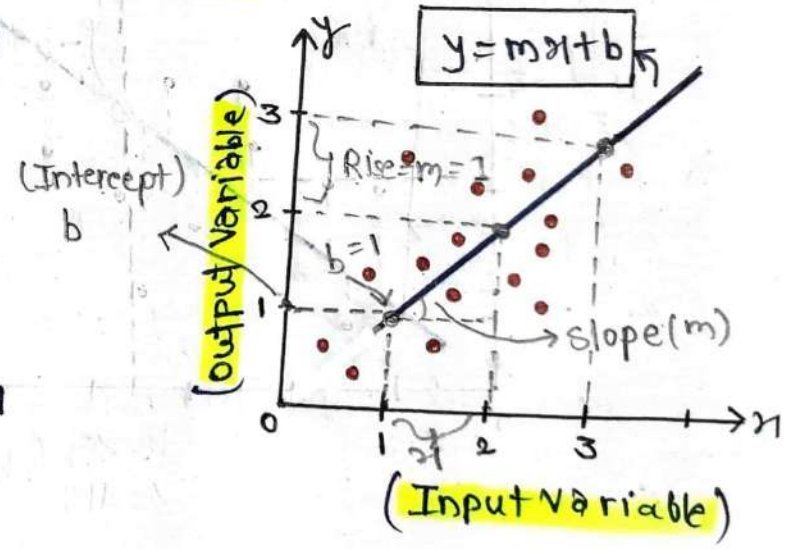
The algorithm draw a straight line through the scatter plot known as Best-Fit Line



- It helps to predict the output value for new input data while minimizing the overall prediction error.

↳ **Mathematical Representation**: The "Best fit line" Represented by the classic linear Regression:

$$y = mx + b$$



- y = Prediction / Pridicted Value
- x = input data point
- m = slope (which dictates how much y changes when x changes by one step)
- b = Intercept (the point where the line crosses the Y-Axis, representing the starting value $x = 0$)

↳ **Hypothesis Notation** : A Mathematical equation used by linear Regression Model to predict the output from the input.

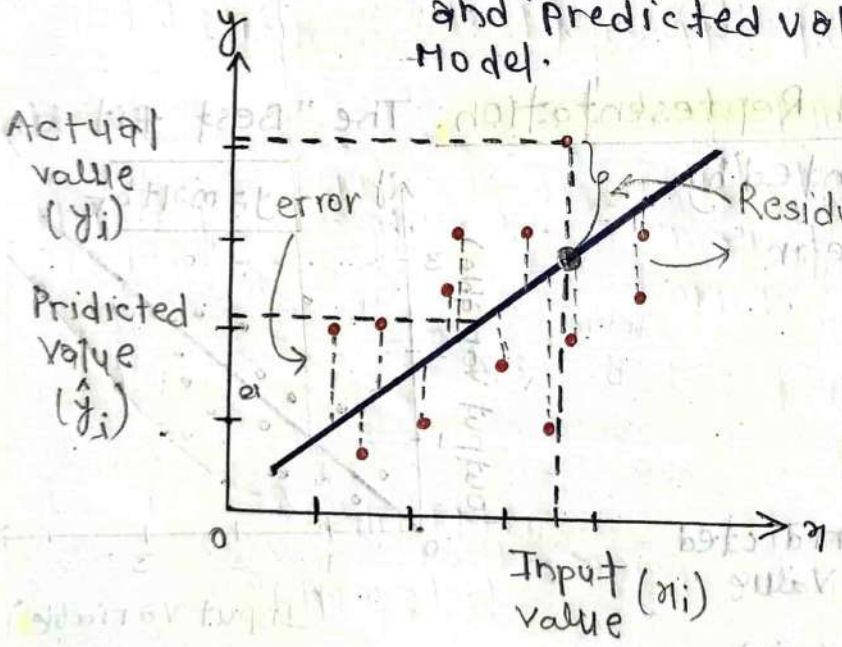
$$h_\theta(x) = \theta_0 + \theta_1 x$$

hypothesis function

- θ_0 = intercept
- θ_1 = slope(m)
- $h_\theta(x)$ = Predicted output (Hypothesis Function)

* If some like → $y = mx + b$; y is $h_\theta(x)$, m is θ_0, x is x, & b is θ_0.

↳ **Residual error** : It is the difference between "Actual value" and "Predicted value" Made by regression Model.



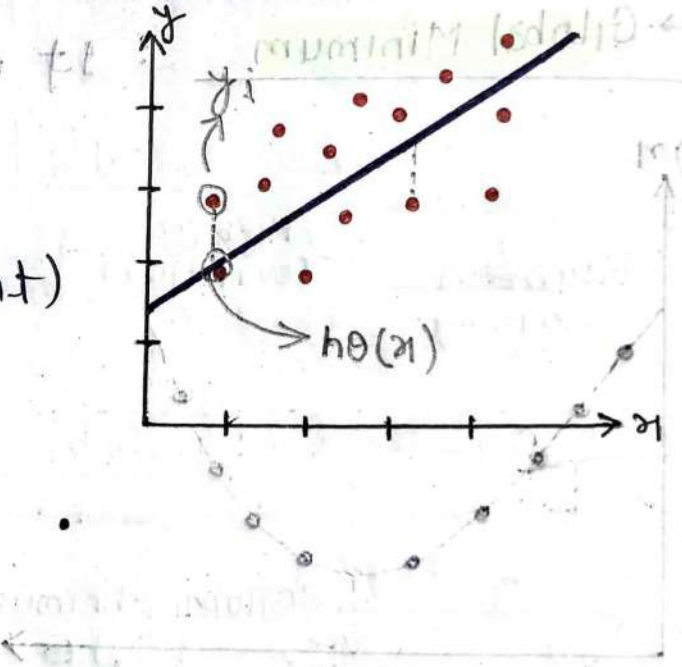
- **Actualvalue (y)** : The real or observed value in the dataset.
  ex - Actual house price = 50 00 000
- **Pridicted value($\hat{y}$)** : The value estimated by the regression model.
  ex - Pridicted house price = 4850 000

→ **Cost Function** : It measures how much error the regression model makes by comparing the predicted values with the actual value.

goal is to find the values of θ_0 & θ_1, that produce the minimum prediction error.

↳ **Formula**

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h\theta(x_i) - y_i)^2$$



- $J(\theta_0, \theta_1)$ = cost function
- θ_0 = Intercept
- θ_1 = slope (Weight / coeffient)
- m = Number of Training examples
- i = Index
- x_i = Input Feature
- y_i = Actual value
- $h\theta(x_i)$ = Hypothesis Function (Prediction)
- $h\theta(x_i) - y_i$ = Error (Residual)

* We have to minimize the cost function by changing the value of θ_0, θ_1

→ Gradient Descent

It is an optimization algorithm used to minimize the cost function

It repeatedly updates θ values until the cost becomes as small as possible.

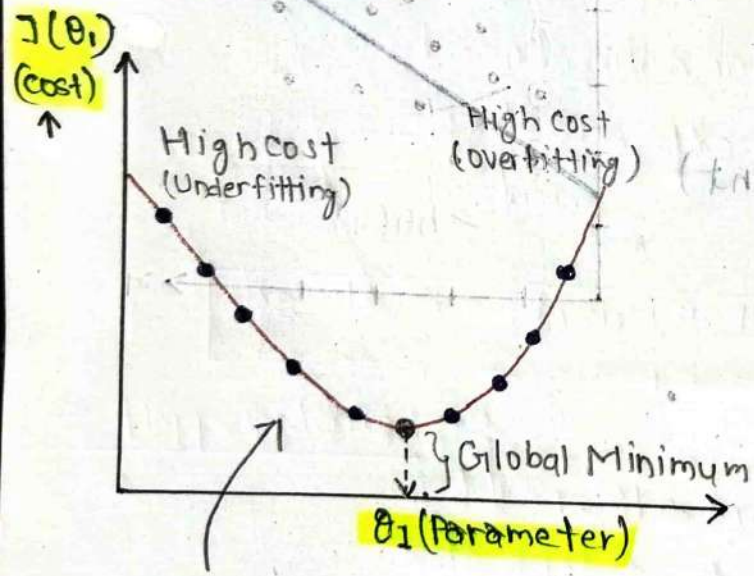
- Update Rule

$$\theta = \theta - \alpha \frac{\partial J}{\partial \theta}$$

- θ = Parameter being updated
- α (alpha) = Learning Rate
- ∂J/∂θ = Gradient (slope of the cost function)

→ Global Minimum

: It is the lowest point of the cost function. at this point, the model has smallest possible prediction error.



Cost funtion curve
(variation in J with θ_1)

→ Cost function curve : helps to measure the error between a model's predicted & actual value.

It looks like U-shaped curve.

For two parameter (θ_0 & θ_1) it forms a 3D bowl-shaped surface.

→ **Convergence** : It means Gradient Descent has reached (or is very close to) the minimum cost function.

The Algorithm stops updating Because further improvement are extremely small.

→ **Learning Rate (α)**

It Determines how large each update step is:

* update step: Process of changing the model parameters (weights) a little bit in the direction that reduces the cost function.

↳ **If α is too small**

- Training Becomes very low
- Needs many iterations

↳ **If α is too large**

- overshoots the minimum
- May oscillate
- May never converge

A good learning rate allows the Model to converge quickly & safely.

→ Simple Linear Regression

It uses one input feature (independent variable) to predict a single output (dependent variable).

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Since there is only one input feature, the model is represented by a straight line on a 2D graph.

→ Multiple Linear Regression

It uses two or more input features to predict a single output.

each input feature has its own weight, allowing the model to learn the contribution of every feature.

$$h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

→ Hyperplane

It is the generalization of straight line to higher-dimensional space

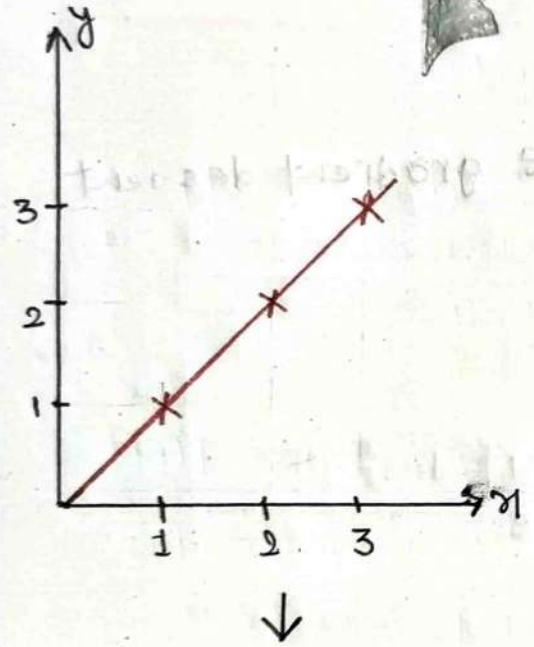
- 1 input feature → straight line (2D)
- 2 input feature → Plane (3D)
- 3 or more input feature → Hyperplane (4D and higher)

→ Cost Function calculation (step by step)

• Training Data

X	Y
1	1
2	2
3	3

$m = 3$



↳ Case : 1 ($\theta_0 = 0$, $\theta_1 = 1$)

$h(\theta) = \theta_0 + \theta_1 x_1$ → hypothesis func

$h(\theta) = 0 + 1(1) = 1$

$h(\theta) = 0 + 1(2) = 2$

$h(\theta) = 0 + 1(3) = 3$

• cost calculation:

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h\theta(x)^{(i)} - y^{(i)} \right)^2$$

$$= \frac{1}{6} \left[(1-1)^2 + (2-2)^2 + (3-3)^2 \right]$$

$$= 0$$

$J(\theta_0) = 0$

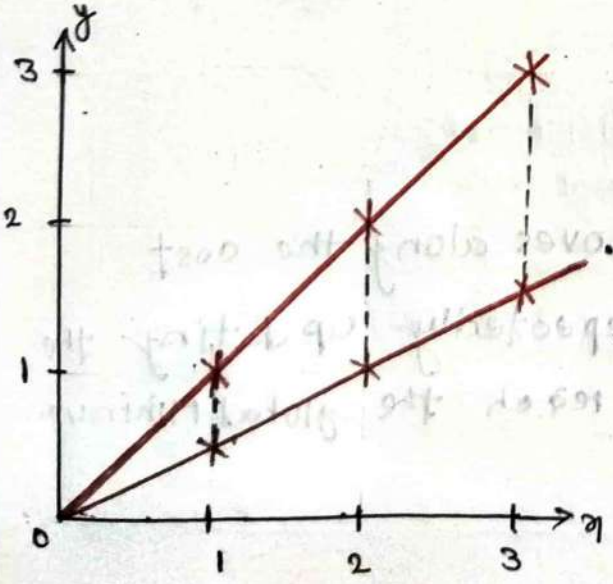
↳ case : 2 ($\theta_1 = 0.5$, $\theta_0 = 0$)

$h(\theta) = \theta_0 + \theta_1 x_1$

$h(\theta) \Rightarrow 0 + .5(1) = .5$

$h(\theta) \Rightarrow 0 + .5(2) = 1$

$h(\theta) \Rightarrow 0 + .5(3) = 1.5$



• cost calculation:

$$J(\theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h\theta(x)^{(i)} - y^{(i)} \right)^2$$

$$= \frac{1}{6} \left[(1-0.5)^2 + (1-2)^2 + (3-1.5)^2 \right]$$

$$\approx 0.58$$

Case : 3 ($\theta_1 = 0, \theta_0 = 0$)

$$h(\theta) = (\theta_0 + \theta_1 x_i)$$

$$= 0 + 0 = 0$$

$$= 0 + 0 = 0$$

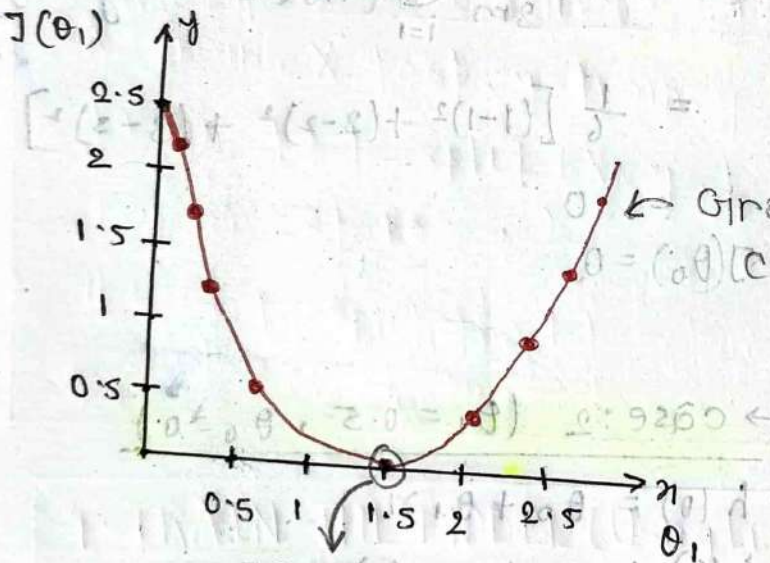
$$= 0 + 0 = 0$$

$$J(\theta_1) = \frac{1}{6}\left[(0-1)^2 + (0-2)^2 + (0-3)^2\right]$$

$$\approx 2.3$$

x	y
1	

- With all three cases we can make gradient descent curve on graph,

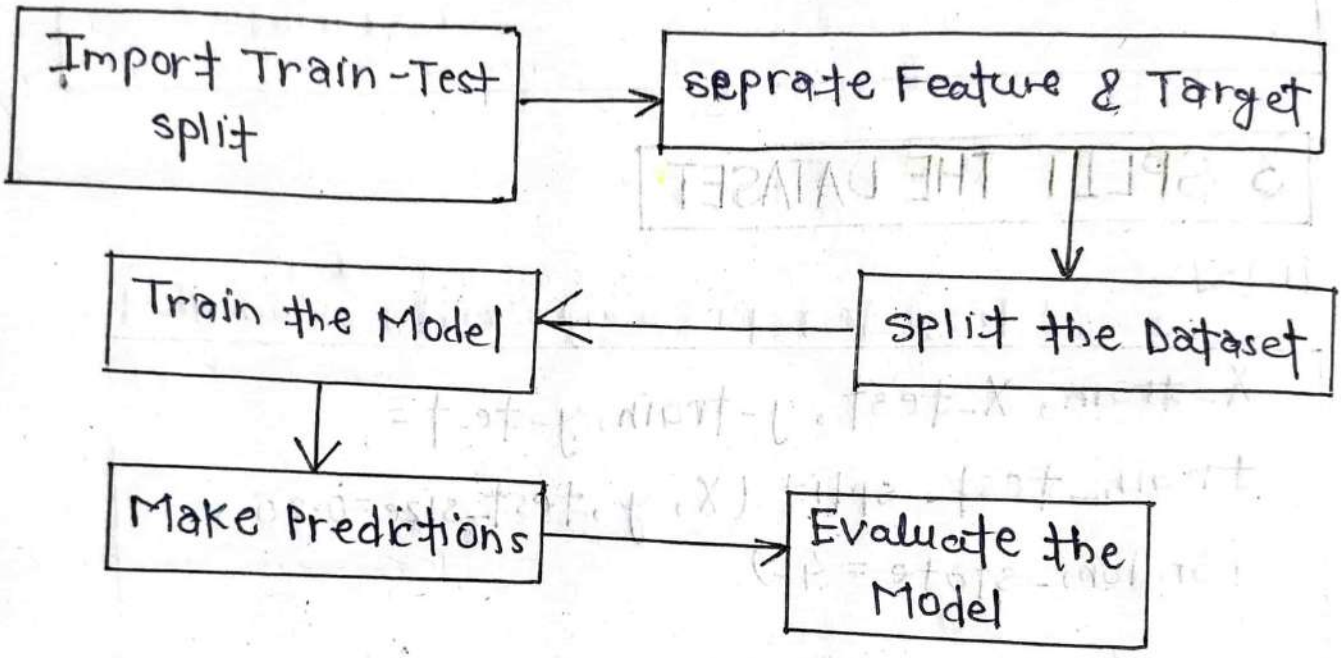


→ **Gradient descent curve** : It moves along the cost function curve by repeatedly updating the model parameter to reach the global minimum.

PROJECT: 1
PREVIOUS PART : INSURANCE PREDICTION PROJECT

In part-I, project-1 we did insurance prediction project till Feature Engineering & Extraction.

Here we perform:



1. IMPORT TRAIN-TEST SPLIT

In [1]

```
from sklearn.model_selection import
train_test_split
#Used to split the data set into training & testing
set:
```

2. SEPRATE FEATURES AND TARGET

In[2]:

```
X = final_df.drop('charges', axis = 1)
# used to store input features (independent variables)
y = final_df['charges']
# used to store the target variable (dependent variable)
```

3. SPLIT THE DATASET

In[3]:
We the type "train_test_split" in cell + shift + tab automatically suggested

```
X_train, X_test, y_train, y_test =
train_test_split(X, y, test_size=0.20,
random_state = 42)
# used to split the dataset into 80% training
  and 20% testing data
```

Just we change anything according to our requirement

4. TRAIN THE MODEL (Using Linear Regression)

In[4]:

```
from sklearn.linear_model import LinearRegression
# to Import the Linear Regression Model.
model = LinearRegression()
# to create the Linear Regression Model.
model.fit(X_train, y_train)
# used to train the Model using the traing data.
```

5. MAKE PREDICTIONS

In [5]:

```
y_pred = model.predict(X_test)
# used to predict insurance charges for
  the test data
```

6. EVALUATE THE MODEL

In [6]:

```
from sklearn.metrics import r2_score
#Used to import the R² score evaluation metric

r2 = r2_score(y_test, y_pred)
# Used to calculate the R² score

r2
# Display R² score

n = X_test.shape[0]
# used to get the Number of test sample

p = X_test.shape[1]
# used to get the number of input
  feature

adjusted_r2 = 1 - ((1- r2) * (n-1) / (n-p-1))
# used to calculate the Adjusted R² score

adjusted_r2
# used to display the Adjusted R² score
```

→ y_test

When Building a ML Model, the dataset is typically split into training data (80%) & testing data (20%)

"y_test" contains the Actual (true) target values from the testing dataset.

↳ Purpose of y_test

After the Model is trained using X_train & y_train it predicts the target values for X_test producing y_pred.

↳ Evaluation

The predicted values (y_pred) are compared with the actual values (y_test) to measure the model's performance using evaluation metrics such as the R^2 score, Mean Absolute Error (MAE), Mean squared Error (MSE) & Root Mean squared Error (RMSE)

↳ Outcome

These performance metrics help determine how accurately the Model predicts unseen data & whether it is ready for deployment.

→ **Performance Metrics** : evaluates how well ML model performs by comparing predicted value with actual value

↳ **Purpose:**

- Measure the Model's accuracy & reliability to Determine if it is suitable for deployment

↳ **common Performance Metrics for Regression:**

(1) R^2 score (coefficient of Determination)

It Measures how well the regression model explains the variation in the target variable.

- **Formula:**

$$R^2 = 1 - \frac{\Sigma (y_i - \hat{y}_i)^2}{\Sigma (y_i - \bar{y})^2}$$

- y_i = Actual value
- $\hat{y}_i$ = Predicted value
- $\bar{y}$ = Mean of actual value
- $\Sigma (y_i - \hat{y}_i)^2$ = Residual sum of squares (Rss)
- $\Sigma (y_i - \bar{y})^2$ = Total sum of squares (Tss)

- **Interpretation**

- $R^2 = 1.0$ (100%) = Perfect prediction
- $R^2 = 0.89$ (89%) → Model explains 89% of variance in the Target variable.
- $R^2 = 0$ → Model performs no better than predicting the mean.
- R^2 can be -ve if model peforms worse than simply predicting the avg target value.

(2) Adjusted R^2

It evaluates the model while accounting for the number of input features. It penalizes unnecessary features that do not improve the model.

- **Formula**

$$\text{Adjusted } R^2 = 1 - \frac{(1-R^2)(n-1)}{n-p-1}$$

- n = no of observations (rows)
- p = no. of independent features

- **Interpretation**

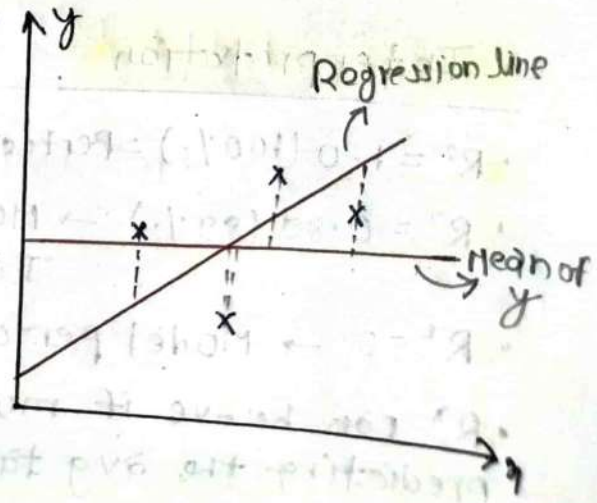
Suppose we build three Models:

Feature	R^2	Adjusted R^2
1	81%	79% — Moderate
2	88%	89% ✓
3	89%	86% ✗

3rd R^2 ↑, Adjusted R^2 ↓, indicating that the additional feature does not improve the model enough to justify its inclusion

↳ **Model selection Rule:**

- R^2 = Measures the Model overall Fit
- Adjust R^2 = compares models with different no. of features
- Choose the Model with the highest Adjusted R^2.



→ Overfitting & Underfitting

A Machine learning model should perform well on both training & testing data. If it fails to generalize to new data, it suffers from overfitting & underfitting.

↳ **Overfitting**: Model performs well on training data but poorly on testing data because it memorizes the training data.

Training → Model perform better (low bias)

Testing → Model not performed good (High variance)

↳ **Underfitting**: Model performs poorly on both training & testing data because data it is too simple

Training → Model perform is bad (high bias)

Testing → Model Not performed good (high variance)

• Ex

Model : 1	Model : 2	Model : 3
Training = 92%	Training = 92%	Training = 68%
Testing = 64%	Testing = 90%	Testing = 64%
↓	↓	↓
overfitting	Perfect Model (low bias) (low variance)	Underfitting
	↓	
	Ridge & Lasso Regression	

Ridge & Lasso Regression

- **Ridge Regression (L2)**: Adds square of the coefficient to the cost function to reduce overfitting.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^i) - y^i)^2 + \lambda \theta_1^2$$

- $J(\theta_0, \theta_1)$ - cost Function
- m = No. of training examples
- i = Training example index
- $h_\theta(x^i)$ = Predicted value
- y^i = Actual value
- θ_1 = Slope
- λ = Regularization Parameter
- $\lambda \theta_1^2$ = Ridge (L2) Regularization Term
- λ = control the amount of regularization

- **Lasso Regression (L1)**: Adds absolute value of the coefficient to the cost function to reduce overfitting & perform feature selection.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x^i) - y^i)^2 + \lambda |\theta_1|$$

- $\lambda |\theta_1|$ = Lasso (L1) regularization Term

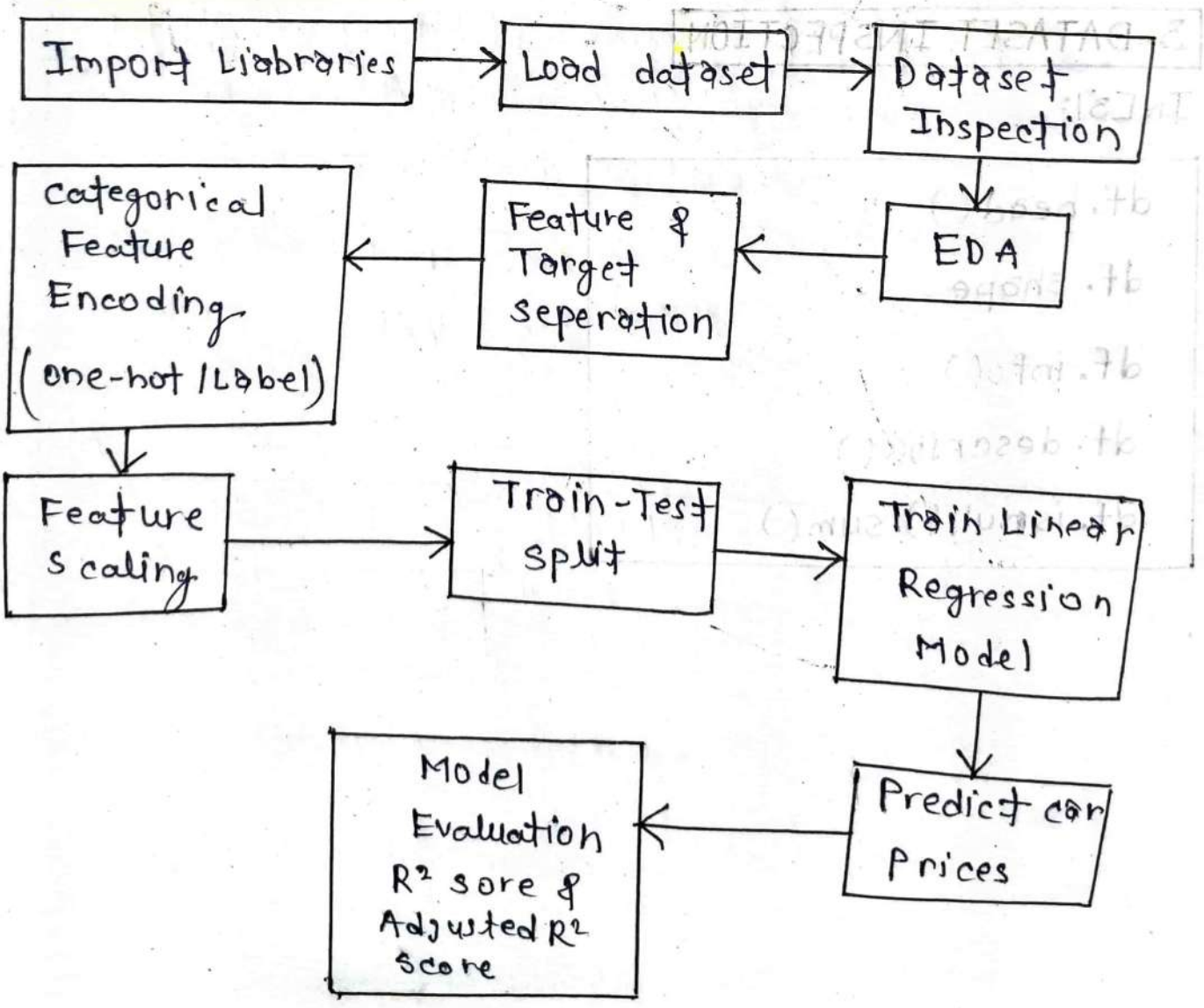
PROJECT : 2
(FORD CAR PREDICTION PROJECT)

→ Overview

The Project uses Linear Regression to predict the selling price to used Ford cars Based on features like model, year, mileage, transmission, fuel type, tax, MPG, and engine size.

It includes EDA, data preprocessing, feature encoding, feature scaling, model training & evaluation.

→ Workflow / pipeline



1. IMPORT LIBRARIES

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

2. LOAD DATASET

In[2]:

```
df = pd.read_csv("ford.csv")
```

3. DATASET INSPECTION

In [3]:

```
df.head()
df.shape
df.info()
df.describe()
df.isnull().sum()
```

4. EDA (EXPLORATORY DATA ANALYSIS)

In [4]:

```
sns.histplot(df['price'], bins=50, kde=True)
# Plot the Distribution of car prices

sns.heatmap(df.corr(numeric_only=True),
annot=True)
# Display correlation between numeric features

sns.boxplot(data=df, x='year', y='price')
# Compare Manufacturing year with car
  price
plt.xticks(rotation=90)
# Rotate X-Axis labels for better readability

sns.scatterplot(data=df, x='mileage',
y='price')
# Display Relationship between mileage &
  price

sns.boxplot(data=df, x='engineSize',
 y='price')
# compare engine size with price

df.columns

sns.boxplot(data=df, x='transmission',
 y='price')
# compare transmission type with price

sns.boxplot(data=df, x='fuelType', y='price')
# compare fuelType with price

sns.boxplot(x=df['model'], y=df['price'])
# compare car model with Price
plt.xticks(rotation=90)
# Rotate x-Axis labels
```

5. FEATURE & TARGET SEPRATION

In [6]:

```
X = df.drop(columns = ['price'], axis = 1)
# Remove the target column 'price' to
  create the feature matrix
y = df ['price']
# store the target variable
X
# Display Matrix feature
df.columns
```

6. ONE HOT ENCODING

In [6]:

```
X_one_encode = pd.get_dummies (X ,
columns = ['model','transmission','fuelType'],
drop_first = True)
# convert categorical columns into dummy
  variables
X_one_encode = X_one_encode.astype(int)
# convert boolean values into integers (0 & 1)
X_one_encode
# Display the encoded dataset
```

* Here we use two encoding one-hot & label, because we get two different dataset from them we make two models & compare them and used it.

7. LABEL ENCODING

In[9]:

```
from sklearn.preprocessing import LabelEncoder
#Import LabelEncoder for converting categorical
  values into numerical labels

columns = ['model', 'transmission', 'fuelType']
#specify the categorical column to encode

Xlable = X.copy()
#create a copy of the original feature dataset
label_encoders = { }
#create a dictionary to store each fitted
 LabelEncoder

for col in columns:
        le = LabelEncoder()
        #create a label Encoder object
        Xlable[col] = le.fit_transform(
        Xlable[col].astype(str))
        #convert categorical values in to
          numerical labels
        label_encoder[col] = le
        #store the fitted encoder
        Xlable
        # Display the lable Encoded dataset
```

8. FEATURE SCALING

In [8]:

```
from sklearn.preprocessing import StandardScaler
# Import standard scaler
numerical_cols = ['year', 'mileage', 'tax', 'mpg',
                  'enginesize']
#select numerical column
Scaler = StandardScaler()
#create a standardscaler object
X_one_encode[numerical_cols]=Scaler.fit_transform(
                X_one_encode[numerical_cols])
#scale numerical columns in one-Hot encoding
 dataset.
X_one_encode
# Display the scaled dataset
Xlable.columns
Xlable[['model','year','transmission','mileage',
        'fuelType','tax','mpg','enginesize']]=
        Scaler.fit_transform(Xlable[['model',
         'year','transmission','mileage',
         'fuelType','tax','mpg','enginesize']])
#Scale the Label Encoded Dataset
Xlable
#Display the scaled Label Encoded dataset
```

* We did Feature scaling for both different Dataset X_one_encode & Xlable for creating to make 2 different Model.

9. IMPORT ML LIBRARIES

In [9]:

```
from sklearn.model_selection import
train_test_split
#Import function to split the dataset
from sklearn.linear_model import
LinearRegression
#Import Linear Regression algorithm
from sklearn.metrics import (
    mean_absolute_error, mean_
    squared_error, r2_score)
#Import regression evaluation metrics
```

10. TRAIN-TEST SPLIT

In [10]:

```
X_train, X_test, y_train, y_test = train_test_split(
  X_one_encode, y, test_size = 0.33,
  random_state = 42 )
#split the onehot encoded dataset into training & testing sets
```

11. LINEAR REGRESSION MODEL TRAINING

In [11]: ↳ for one hot encoded data

```
model = LinearRegression()
#create a Linear Regression model
model.fit (X_train, y_train
#Train the model using the training dataset
```

12. PREDICTION

In [12]:

```
y_pred = model.predict(X_test)
# Predict prices for the testing dataset
y_pred
# Display predicted values
y_test
# Display actual values
```

13. MODEL EVALUATION

In [13]:

```
r2 = r2_score(y_test, y_pred)
# Calculate the R² score
r2 → r2 value of Model is out[13]: 0.83 or 83%
#Display R² score
```

```
n = X_test.shape[0]
# No. of test samples
p = X_test.shape[1]
# No. of input features
adjusted_r2 = 1 - ((1-r2) * (n-1)) / (n-p-1)
#calculate Adjusted R² score
print("Adjusted R² score:", adjusted_r2)
```

14. LINEAR REGRESSION USING LABLE ENCODED DATA

In [14]:

```
X_train, X_test, y_train, y_test = train_test_split(
 Xlable, y , test_size=0.33, random_state=42)
#split the Label Encoded Dataset
model2 = LinearRegression()
# create new Linear Regression model name 'model2'
model.fit (X_train, y_train)
#train the model
```

```
y_pred = model2.predict(X_test)
# Predict price
y_pred
#Display Price
y_test
# Display actual values
r2 =r2_score(y_test, y_pred)
# calculate R² score
r2  → out [14]: 0.73 / 73%
#Display R² score
```

→ **Result:**

- Model (one-Hot encoding) : R^2 score = 0.83
- Model2 (Label encoding) : R^2 score = 0.73

Best Model is = Model (one hot encoding)
Because it achieved higher prediction accuracy.

PART-3

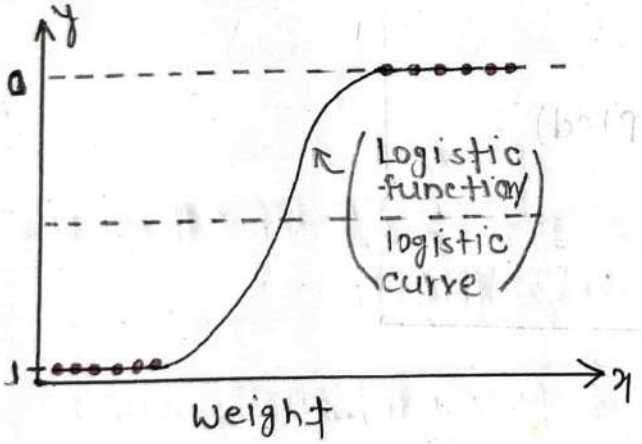
(CLASSIFICATION ALGORITHMS (SUPERVISED LEARNING))

→ **Classification** : It is a supervised learning algorithm used to predict categorical (discrete) value

ex- • spam or Not spam
• Disease or No disease
• customer will buy or Not
• Titanic : survived (1) or Not survived (0)

output is always from predefined classes.

→ **Logistic Regression** : It is a classification algorithm used to predict the probability of binary outcomes (0 or 1).



ex =

• spamDetect: spam (1) or Not spam
• Exam Result = Pass (1) or Fail (0)
• Weight & obesity

Weight	Obesity
92	yes (1)
63	No (0)
24	No (0)
102	yes (1)
196	yes (1)
⋮	

→ Why Not Linear Regression ?

In Linear Regression can produce value like: -2 , 3.5 , 10

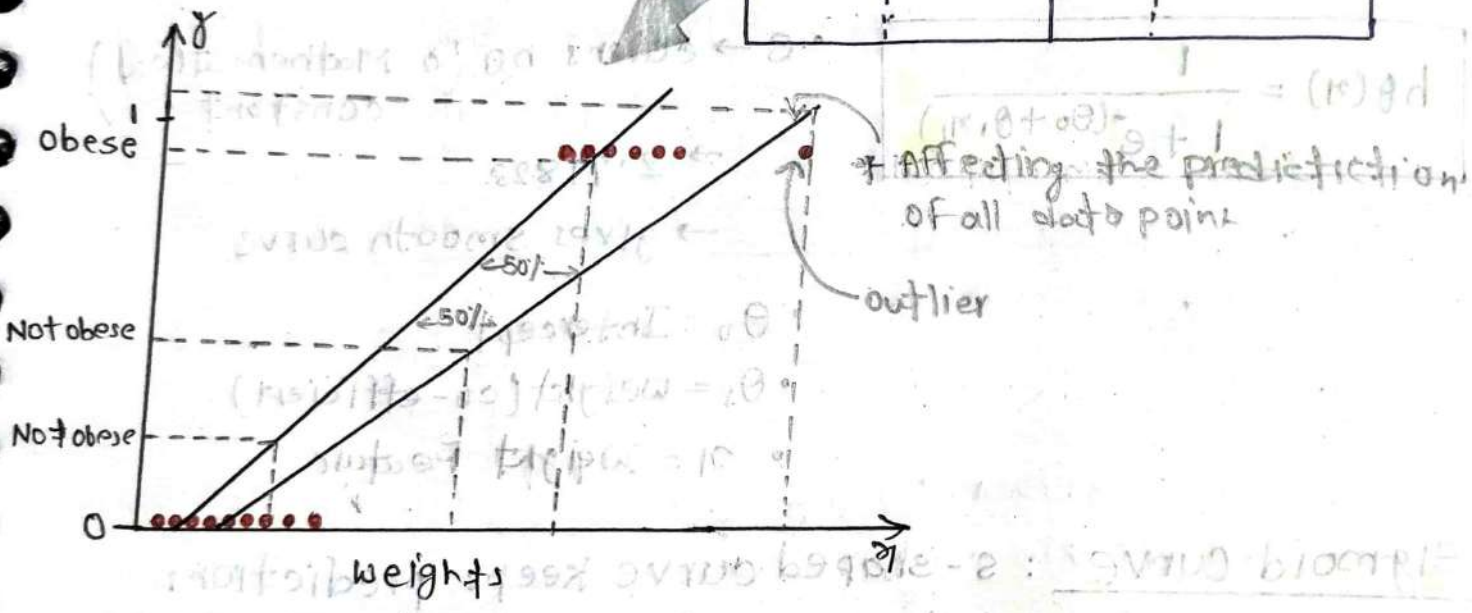
These values are invalid as probability because must lie between 0 & 1.

Ex:

If we use linear Regression for this data, it fits a straight line through the data

Weight	obesity
92	yes
63	no
24	no
102	yes
196	yes
268	yes

outlier →



Linear regression cause Major Problems:

(1) **Outliers affects the Best fit line** : Extreme data points, like (outliers), can pull the straight line up or down, leading to poor predictions.

(2) It can produce Negative or very large positive values, making it unsuitable for classification,

or etc More problems

* So do prevent all of these things we use squashed line → "s-shaped line"

* To make the straight into s-shaped curve we will use "SIGMOID" activation function.

→ **Sigmoid (Logistic) Function**: The function converts any real number into a probability between 0 & 1.

- **Formula**

$$h_\theta(x) = \frac{1}{1 + e^{-(\theta_0 + \theta_1 x_1)}}$$

- e → eulers no (a Mathematical constant)
  - → 2.71828
  - → gives smooth curve
- θ_0 = Intercept
- θ_1 = weight (co-efficient)
- x = Input Feature

- **Sigmoid curve** : s-shaped curve keeps predictions between 0 & 1.

- **Decision Boundary** : It is a threshold or Boundary that seprates data points into different classes based on the model's predicted probability or output.

After calculating probability:

- Probability ≥ 0.5 → class = 1
- Probability < 0.5 → class = 0

It can be changed according to requirement.

→ Cost Function (Logistic Regression)

It Measures how wrong the model's predictions are. It helps the model to improve by minimizing the pridiction error during training. In logistic it use Log loss function.

- Formula

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log\left(h_\theta(x^i)\right) + (1-y^i)\log\left(1-h_\theta(x^i)\right)\right]$$

- m = No. of training examples
- y = Actual class (0 or 1)
- $h\theta(x)$ = Predicted probability

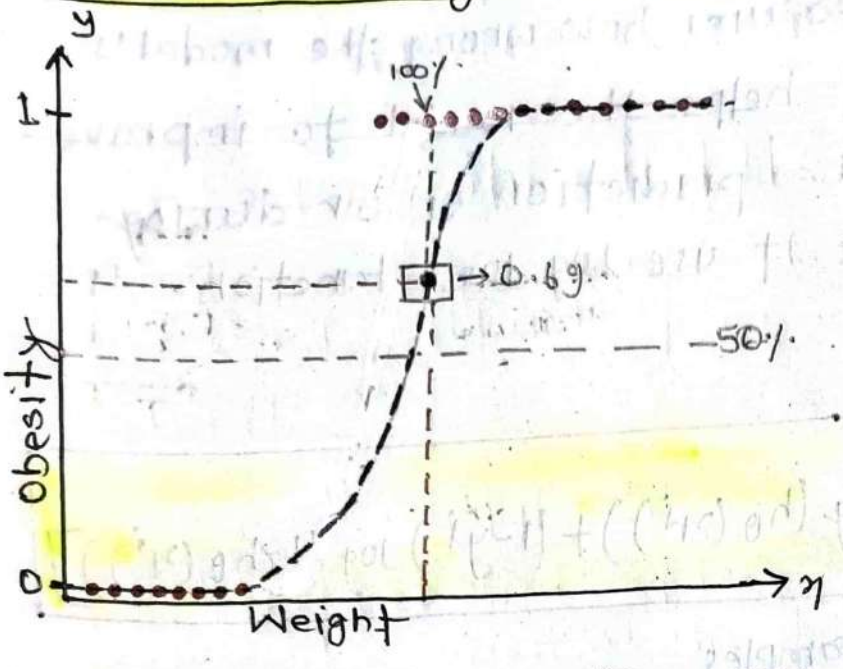
→ Linear Regression Vs Logistic Regression

	Linear Regression	Logistic Regression
output	(1) continuous values	(1) class probabilities (0-1)
Prediction	(2) Any real No.	(2) Between 0 & 1
Graph	(3) straight line	(3) s-shaped (sigmoid)
Cost Function	(4) Mean squared Error (MSE)	(4) Log loss (Binary cross-Entropy)
Examples	(5) House price, salary Prediction	(5) Spam Detection, Disease Prediction, Titanic survival
	(5) θ_0, θ_1 → values → to create best fit line	(5) θ_0, θ_1 → values → to create s-shape line

→ Log loss (Binary cross - Entropy)

It's a loss function used in Logistic Regression.

It Measures how different the predicted probability is from the actual class (0 or 1)



- Lower = Better loss = Prediction
- Higher = Worse Loss = Prediction

ex:- * Predicted probability = 0.99
and actual is one (1)
↳ very less loss

* predicted probability = 0.01
& actual No. is 1
↳ very high less

⇓

Mean "Penalty" in this algorithm

⇓

Penalty = is calculated with "log loss function".

• Formula

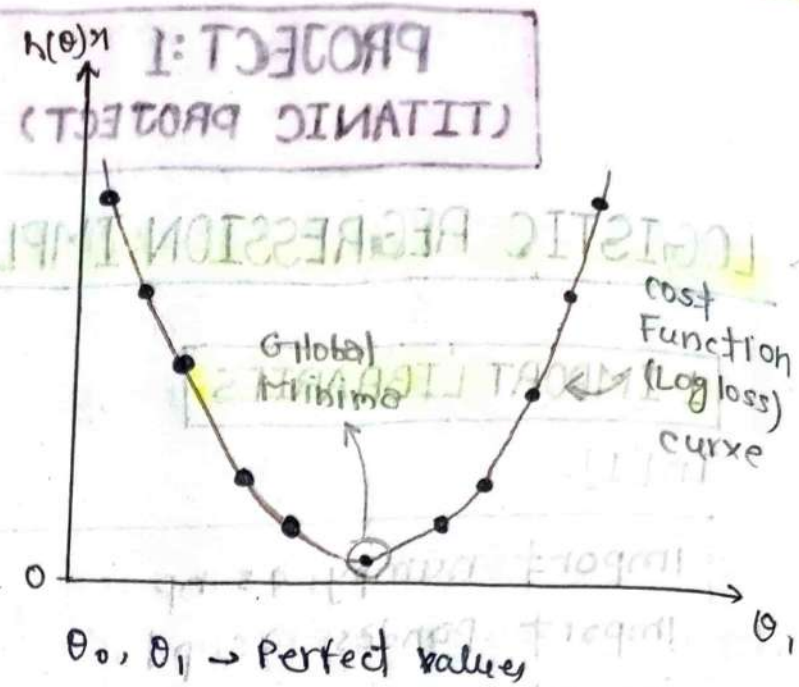
$$\text{Log loss} = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{y}^{(i)}) + (1-y^{(i)})\log(1-\hat{y}^{(i)})\right]$$

- m = no. of sample
- y^i = actual label (0,1)
- $\hat{y}^i$ = Predicted Probability (0 -1)

ex - If $y^{(i)} = 1$

$\hat{y}^{(i)} = 0.54$

log loss = 0.616



θ_0, θ_1 → Perfect values

→ **Gradient Descent** : After calculating the log loss, "Gradient Desent" updates the parameter θ_0 & θ_1 repeatedly to reduce the loss.

or it moves step by step down this U-shaped curve until it reaches the global Minimum, giving the optimal value of θ_0 & θ_1.

PROJECT:1
(TITANIC PROJECT)

→ LOGISTIC REGRESSION IMPLEMENTATION

1. IMPORT LIBRARIES

In[1]:

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
```

2. LOAD DATASET

In[2]:

```
df = sns.load_dataset("titanic")
```

3. EDA

In[3]:

```
df.head()
df.columns
df.info
df.describe
df.shape
df.isnull().sum()
df.duplicated().sum()
```

4. DATA CLEANING

In[4]:

```
df.drop( ["deck","embark_town","alive","class","who",
          "adult_male"], axis = 1, inplace =True)
#Remove unnessary columns
df.info
#Verify Updated dataset
df ["age"].fillna(df["age"].mean(), inplace =True)
#Replace missing Age values with the mean age
df.dropna(subset = ["embarked"], inplace = True)
# Remove rows with missing Embarked values.
df.info
# check updated dataframe
```

5. FEATURE & TARGET SELECTION

In[5]:

```
df =df.astype(int)
# converts all columns to integer datatype
df.head
# for preview of processed data
X = df.drop ("survived", axis = 1)
#Create the feature matrix (input variables) & dropping
  survived column because it is target variable
y= df["survived"]
# create the target variable (output)
```

6. TRAIN-TEST SPLIT

In [6]:

```
from sklearn.model_selection import train_test_split
# Import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
  X, y, test_size = 0.2, random_state = 42)
```

7. BUILD & TRAIN LOGISTIC REGRESSION MODEL

In [7]:

```
from sklearn.linear_model import LogisticRegression
# Import Logistic Regression
model = LogisticRegression()
# Create the logistic Regression model
model.fit(X_train, y_train)
# Train the Model using the training data
```

8. MAKE PREDICTIONS

In [8]:

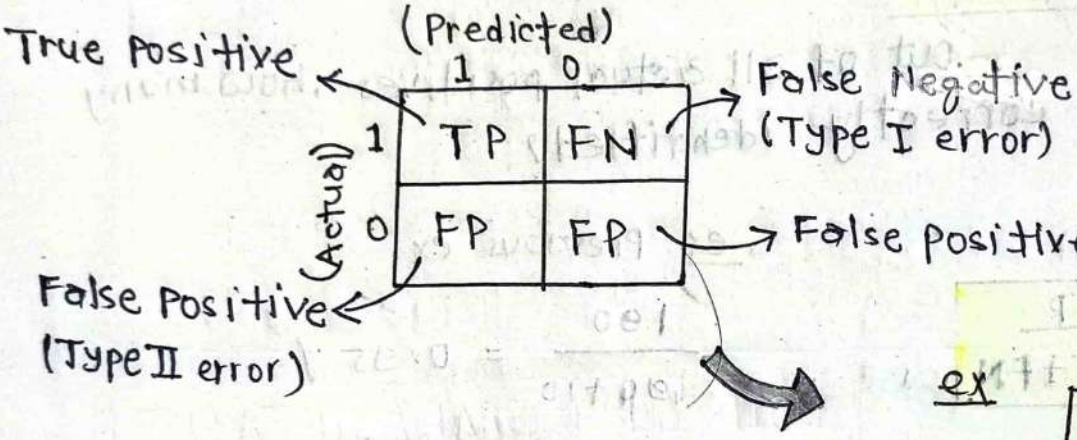
```
y_pred = model.predict(X_test)
# Predict the target values for the test data
y_pred
# Display predicted values
y_test
# Display actual values
```

→ Model Evaluation

It is the process of Measuring how well a machine learning model performs on unseen data.

↳ Confusion Matrix :

It is the table that compares actual and predicted classes to evaluate the performance of the classification model.



ex

190 (TP)	10 (FN)
10 (FP)	40 (FP)

↳ Some Evaluation Metrics derived from confusion Matrix

They are used to measure the performance of a classification Model.

(1.) Accuracy : The percentage of all predictions that are correct.

- **Formula:**

$$\frac{TP + FP}{TP + FP + FN + FP}$$

ex: Previous ex

$$\frac{190 + 40}{190 + 10 + 10 + 40} = 0.92$$

$$= 92\%$$

(2) **Precision**: out of all predicted positives, how many were actually positive?

• Formula

$$\text{Precision} = \frac{TP}{TP+FP}$$

ex: previous ex

$$\frac{190}{190+10} = 0.95\%$$

$$= 95\%.$$

* Use when False positive (FP) are costly (spam Detection)

(3.) **Recall (sensitivity)**

Out of all actual positives, how many were correctly identified?

• Formula

$$\text{Recall}: \frac{TP}{TP+FN}$$

ex: Previous ex

$$\frac{190}{190+10} = 0.95\%$$

$$= 95\%$$

* use when False N are costly (disease prediction)

(4.) **F1-score**

The Harmonic mean of Precision & Recall

A Type of average that gives more Importance to smaller value like balancing Precision & Recall

• Formula

$$F1 = \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

* Use when we need a balance Between Precision & Recall especially for imbalanced datasets.

ex - Previous ex

$$\frac{2 \times 0.95 \times 0.95}{0.95 + 0.95} = 0.95$$

$$= 95\%$$

PROJECT:1
(TITANIC)

→ MODEL EVALUATION IMPLEMENTATION

9. MODEL EVALUATION

In[9]:

```
from sklearn.metrics import accuracy_score,
confusion_matrix, classification_report
# Import evaluation metrics

accuracy_score (y_test, y_pred)
# Calculate the overall prediction accuracy

confusion_matrix (y_test, y_pred)
# Generate the confusion Matrix (Actual Vs Predicted)

print (classification_report (y_test , y_pred))
# Display Precision, Recall, F1 score & support
  we use "print" because it makes it in structure
  with perfect spacing.

df
# Display the find processed dataset
```

→ K-Nearest Neighbours (KNN)

Algorithm that classifies a new data point as new data point by measuring its distance from nearby data points & assigning the class most common among its K-nearest neighbours.

Since KNN is a distance based algorithm, numerical features should be scaled & categorical features should be encoded befor training.

↳ Steps

(1) choose the value of K like - (K=1), (K=7), (K=3), (K=5)
→ Hyperparameter

(2) calculate the distance (usally Euclidean Distance)

(3) sort all distance

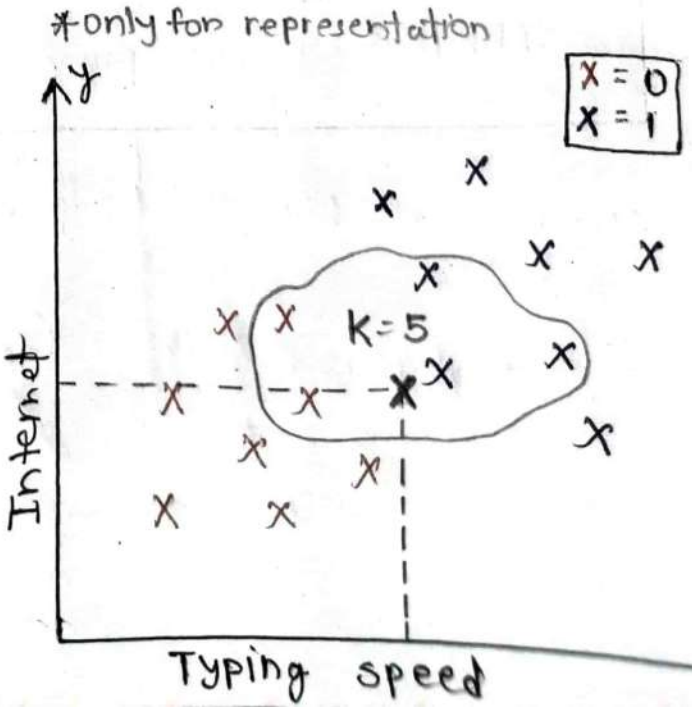
(4) select the K nearest neighbours

(5) Predict the class by majority voting.

Example:

Internet	Typing speed	Hacker
24	56	1
12	22	0
⋮	⋮	⋮
⋮	⋮	⋮
⋮	⋮	⋮

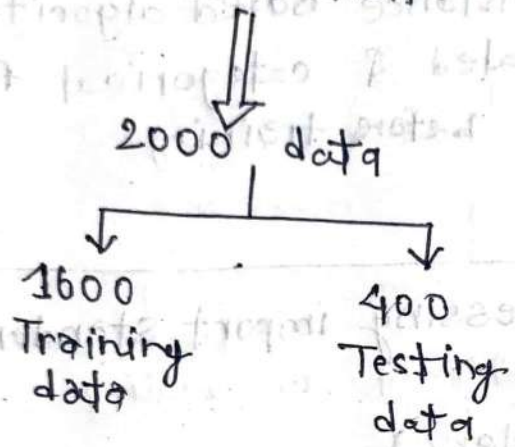
K=5



→ How to K = ? value

- K → it must be can odd Number
- Experiment: check by using different values of K, which performs good use it

cross validation



K = 5 = 86 %

K = 3 = 89 %

K = 1 = 60 %

⋮

K = 25 = 92 % ✓ → Best performing we keep it

- Small k → sensitive to noise (high variance)
- Large k → smoother decision boundary (may underfit)

check first do cross validation.

PROJECT:1
(TITANIC)

→ KNN IMPLEMENTATION

10. FEATURE SCALING: (For KNN)

↳ Because KNN is a distance based algorithm, numerical features should be scaled & categorical feature should be encoded before training.

In [10]:

```
from sklearn.preprocessing import StandardScaler
#Import StandardScaler for feature scaling
scaler = StandardScaler()
# create a standardScaler object
X_train_scaled = scaler.fit_transform(X_train)
# Learn the mean & standard deviation from the
training data then scale it -
X_test_scaled = scaler.transform(X_test)
```

11. BUILD & TRAIN KNN

In [11]:

```
from sklearn.neighbors import KNeighborsClassifier
#Import the KNN classifier
knn_model = KNeighborsClassifier(n_neighbors = 5)
# create a KNN Model with k=5   ← neighbors
knn_model.fit(X_train_scaled, y_train)
```

12. MAKE PREDICTIONS FOR KNN

In [12]:

```
y_pred_knn = knn_model.predict(X_test_scaled)
# Predict the target values for the test data
```

13. MODEL EVALUATION FOR KNN

In [13]:

```
accuracy_score(y_test, y_pred_knn)
# calculate the overall prediction accuracy
confusion_matrix(y_test, y_pred_knn)
# Generate the confusion matrix (Actual V Predicted)
print(classification_report(y_test, y_pred_knn))
#Display Precision, Recall, F1 score & support
```

→ Naive bayes

The Algorithm is based on Bayes Theorem, which predicts a class using probability while assuming that all input features are independent of each other.

• **Bayes Theorem**:- It is a formula used to calculate the probability of an event given that another event has already occured.

$$P(A/B) = \frac{P(B/A) \times P(A)}{P(B)}$$

↳ Independent events

occurance of one event does not affect the Probability of another

$$P(A \text{ and } B) = P(A) \times P(B)$$
or
$$P(A \cap B)$$

ex- Tossing coin, rolling dice

↳ Dependent Events

occurance of one event changes the probability of another

ex- drawing cards without replacement.

↳ conditional Event

It is the probability of an event occuring given that another event has already occured.

$$P(A \text{ and } B) = P(A) \times P(B/A)$$

- Probability of A = P(A)
- Probability of B given A = P(B/A)

↳ Derivation of Bayes Theorem

$$P(A \text{ and } B) = P(B \text{ and } A)$$

$$P(A) \times P(B/A) = P(B) \times P(A/B)$$

$$P(A/B) = \frac{P(A) \times P(B/A)}{P(B)} \longrightarrow \text{Bayes Theorem}$$

↳ Naive Assumption

"Naive" means that algorithm assumes that all input features are independent, even though this may not be true in real-world data

↳ Prediction

Model calculates the probability for each class & predicts the class with the highest probability.

PROJECT : 1
(TITANIC)

→ IMPLEMENTATION OF NAIVE BYES

14. BUILD & TRAIN GAUSSIAN NAIVE BYES

In [14]:

```
from sklearn.naive_bayes import GaussianNB
#Import the gaussian Naive Bayes
model_NB = GaussianNB()
#create a Gaussian Naive Bayes Model
model_NB.fit(X_train, y_train)
#Train the model using the training data
```

15. MAKE PRIDICTION FOR NAIVE BAYES

In [15]:

```
y_pred_NB = model_NB.predict(X_test)
#Predict the target values for the test data
y_pred_NB
#Display the predicted values
```

16. MODEL EVALUATION FOR NAIVE BYES

In [16]:

```
accuracy_score(y_test, y_pred_NB)
#calculate the overall prediction accuracy
confusion_matrix(y_test, y_pred_NB)
#Generate the confusion Matrix (Actual Vs Predicted)
print(classification_report(y_test, y_pred_NB)
#Display Precision, Recall, f1 score & support
```

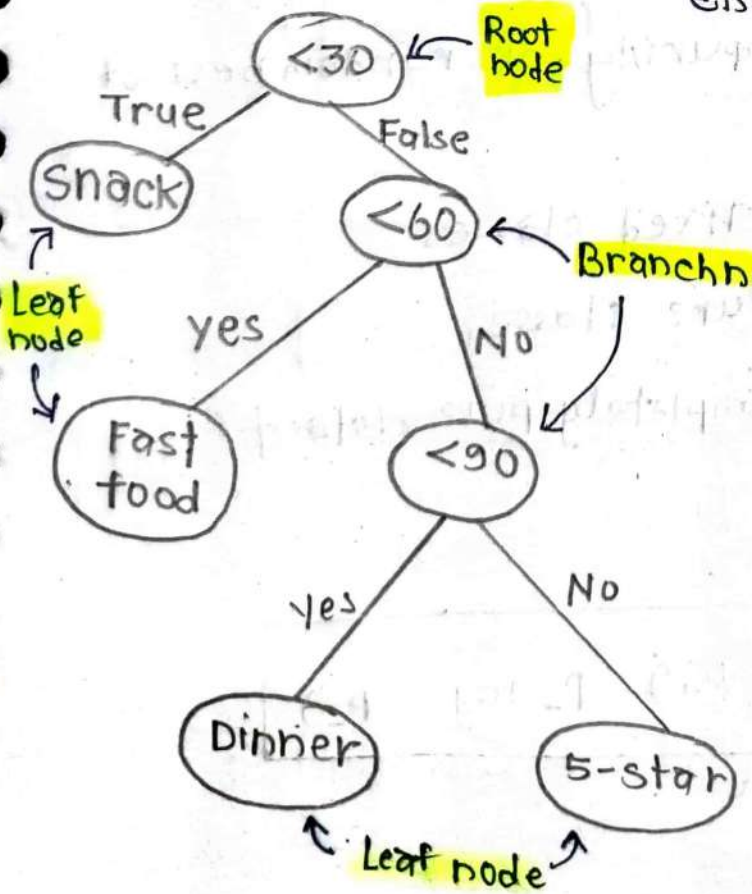
→ Decision Tree

This algorithm is used for classification & regression.

It predicts the output by recursively splitting the data into branches based on the feature that best separates the data until a final prediction (leaf node) is reached

ex:-

```
Money = ?
If money < 30:
    snack
elif money >30 & < 60:
    Fast food
elif money >60 <90:
    Dinner
else:
    5 star
```



↳ Components

- **Root Node** - The first node that represent the best feature used to split the dataset
- **Branch**: A path representing the outcome of decision or condition.
- **Leaf Node**: The final node that gives the predicted class or value.

↳ Steps to Build a Decision Tree

(1) start with the dataset

(2) calculate Entropy & Information Gain for each feature.

(3) choose the feature with the highest information gain as the Root Node.

(4) split the dataset into branches.

(5) Repeat the process recursively for each branch.

(6) stop when:

- All samples belong to the same class (Pure Node), or
- Maximum depth is reached, or
- No more useful splits are possible.

↳ Entropy: Measures the impurity or randomness of a dataset.

- High Entropy → Mixed classes
- Low Entropy → Pure classes
- Entropy = 0 → completely pure dataset

• Formula

$$\text{Entropy}(s) = -P_+ \log_2(P_+) - P_- \log_2(P_-)$$

P_+ = Positive class probability
P_- = Negative class Probability

↳ **Information Gain:** Measure how much a feature reduces entropy after splitting the dataset.

The feature with the "highest Information gain" is selected as the Root Node.

• feature

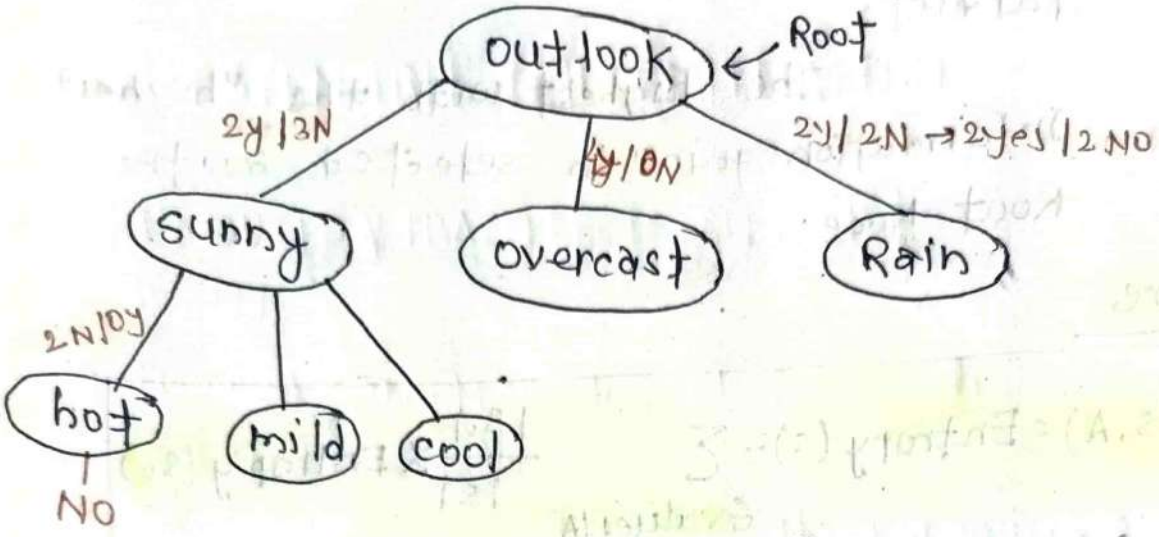
$$IG(S,A) = Entropy(S) - \sum_{\in values(A)} \frac{|S_v|}{|S|} \times Entropy(S_v)$$

Example

Outlook	Temperature	Humidity	Wind	Play
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
overcast	Hot	High	Weak	yes
Rain	Mild	High	Weak	yes
Rain	cool	Normal	Weak	yes
Rain	cool	Normal	Strong	No
Overcast	cool	Normal	strong	yes
Sunny	Mild	High	Weak	No
Sunny	cool	Normal	Weak	yes
Rain	Mild	Normal	weak	yes
Sunny	Mild	Normal	strong	yes
overcast	Mild	High	strong	yes
overcast	Hot	Normal	Weak	yes
Rain	Mild	High	strong	No

↳ solution

Decision Tree



This will go further & further

↳ Calculation

$$entropy(s) = -P_+ \log_2(P_+) - P_- \log_2(P_-)$$

Play = 5NO, 9yes

$$\Rightarrow entropy = \frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right)$$

$$= 0.940$$

outlook =

→ Sunny = 2yes/3NO

$$enropy = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right)$$

$$= 0.971$$

→ overcast = 0

→ Rain = 0.971

Now

Information gain

$$IG(S, outlook) = entropy(S) - \sum_{\epsilon values} \frac{|S_v|}{|S|} Entropy(S_v)$$

$$= 0.941 - \left[\frac{5}{14} \times 0.971 + \frac{4}{14} \times 0 + \frac{5}{14} \times 0.971\right]$$

outlook (IG) = 0.246

- **Information gain**

outlook = 0.246

Temp = ~0.0249

Humidity ~ 0.151

wind ~ 0.048

so, outlook has highest Information gain will take root node

Important Note:- Entropy & Information Gain are calculated recursively at every node (branch) to select the best feature for the next split until the tree reaches a stopping condition

PROJECT:1 (TITANIC)

→ DECISION TREE IMPLEMENTATION

17. BUILD & TRAIN DECISION TREE

In[17]:

```
from sklearn.tree import DecisionTreeClassifier
# Import Decision Tree
model_DT = DecisionTreeClassifier(random_state=42)
# create a Decision tree
model_DT.fit(X_train_scaled, y_train)
# Train the Model using the scaled training data
```

18. MAKE PREDICTIONS FOR DECISION TREE

In[18]:

```
y_pred_DT = model_DT.predict(X_test_scaled)
# Predict the target values for the test data
y_pred_DT
#Display predicted values
```

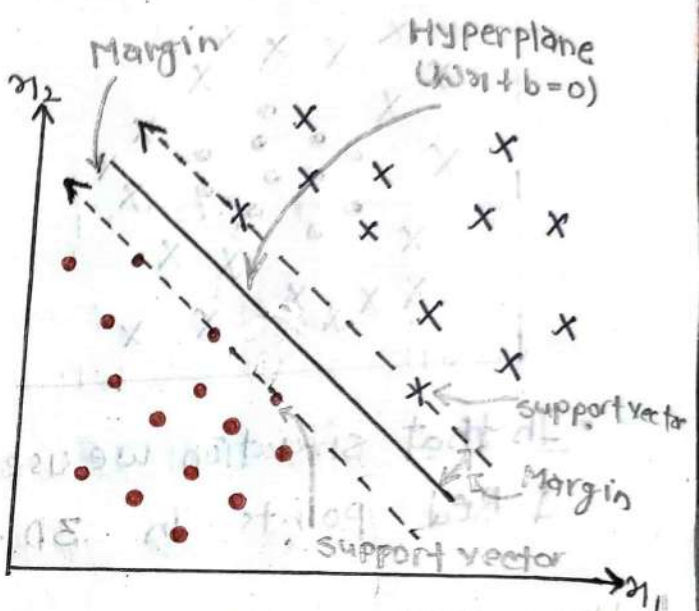
19. MODEL EVALUATION FOR DECISION TREE

In[19]:

```
accuracy_score(y_test, y_pred_DT)
# calculate overall prediction accuracy
confusion_matrix(y_test, y_pred_DT)
# Generate the confusion Matrix (Actual Vs Predicted)
print(classification_report(y_test, y_pred_DT))
#Display Precision, Recall, F1 score, & support
```

→ SVM (support Vector Machine)

It is used for classification and regression. It finds the optimal decision boundary (hyperplane) that maximizes the margin between different classes for accurate prediction.



↳ Key components:

(1.) Hyperplane : It is the Decision boundary that seprates different classes.

- **Equation**

$$Wx + b = 0$$

- x = Input datapoint (Point we won't to predict)
- W = Weight (Determines orientation)
- b = Bias (shifts the hyperplane)

(2.) Margin : It is the distance between the hyperplane and the nearest data points from each class.

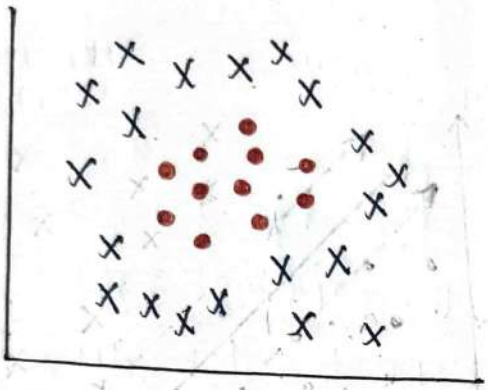
- Maximum Margin = Better Generalization

(3.) Support Vectors:

These are the data points closest to the hyperplane that determine its optimal position.

(4.) **Kernals:** It is a mathematical function that transforms non-linearly separable data into a higher-dimensional space, allowing SVM to find a linear separating hyperplane

ex



- In that situation we use kernal to disting b/w Blue & Red points in 3D.

↳ How SVM works

(1.) Find best hyper plane.
(2.) Maximize the margin
(3) Use support vectors to determine the hyperplane.
(4) If the data is not linearly seperable, apply a kernal function.
(5) Predict the class based on which side of the hyperplane the new data point lies

↳ common Applications

- Image classification
- Face Recognition
- Spam Email Detection
- Text classification
- Sentiment Analysis
- Medical Diagnosis
- gene classification

PROJECT:1
(TITANIC)

→SVC IMPLEMENTATION

20. BUILD & TRAIN SUPPORT FOR SVM

In [20]

```
from sklearn.svm import SVC
#Import the support vector Machine classifier
model_svm = SVC(kernal='rbf')
#Train the model using the scaled training data
model_svm.fit(X_train_scaled, y_train)
#Train the model using the scaled training data
```

21. MAKE PREDICTIONS FOR SVM

In [21]:

```
y_pred_svm_rbf = model_svm.predict(X_test_scaled)
# predict the target values for the test data
y_pred_svm_rbf
# Display the predicted values
```

22. MODEL EVALUATION FOR SVM

In [22]:

```
accuracy_score(y_test, y_pred_svm_rbf)
# calculate the overall prediction accuracy
confusion_matrix(y_test, y_pred_svm_rbf)
# Generate the confusion matrix (Actual Vs Predicted)
print(classification_report(y_test, y_pred_svm_rbf))
# Display Precision, Recall, F1 score, & support
```

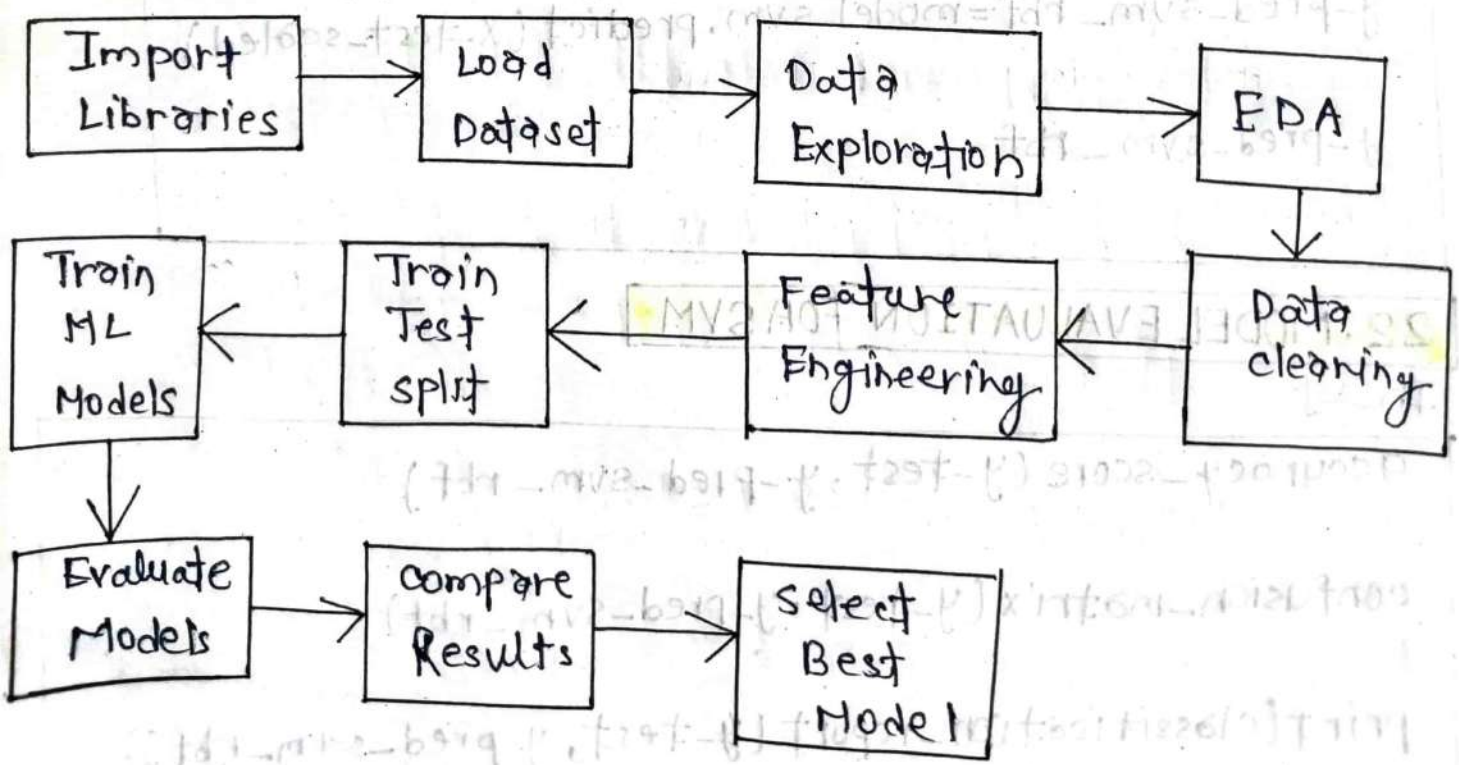
PROJECT: 2
(HEART DISEASE PRIDICTION)

Just like in "part-I", Project-2. we did the same project till using Random Forest but here we did different.

↳ Overview

This project predicts heart disease using Machine learning. The dataset is preprocessed through data cleaning, EDA, encoding & feature scaling. Multiple classification models are trained and evaluated using Accuracy and F1-score to identify the best-performing model.

↳ Workflow



1. IMPORT LIBRARIES

In [1]

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings ('ignore)
```

2. EDA

In [2]

```
df=pd.read_csv("heart.csv")
df.head
df.shape
df.info
df.describe
df.duplicated().sum()
df.isnull().sum()
```

3. TARGET VARIABLE ANALYSIS

In [3]:

```
df['HeartDisease'].value_counts().plot(kind='bar')
```

4. DISTRIBUTION ANALYSIS

In [4]:

```
def plotting(var, num):
    plt.subplot(2, 2, num)
    sns.histplot(df[var], kde=True)
plotting("Age", 1)
plotting("RestingBP", 2)
plotting("cholesterol", 3)
plotting("MaxHR", 4)
plt.tight_layout()
```

5. CHOLESTROL & RESTING BP CLEANING

In [5]:

```
cholesterol_mean = df.loc[df['cholesterol'] != 0,
'cholesterol'].mean()
df['cholesterol'] = df['cholesterol'].replace(0,
cholesterol_mean)
df['cholesterol'] = df['cholesterol'].round(2)
resting_bp_mean = df.loc[df['RestingBP'] != 0,
'RestingBP'].mean()
df['RestingBP'] = df['RestingBP'].replace(0,
resting_bp_mean)
df['RestingBP'] = df['RestingBP'].round(2)
```

6. FEATURE DISTRIBUTION & CORRELATION

In [6]:

```
sns.countplot(x=df['sex'])
sns.countplot(x=df['chestPainType'], hue=df['HeartDisease'])
sns.countplot(x=df['FastingBS'], hue=df['HeartDisease'])
x = 'HeartDisease', y='Cholesterol', data=df)
sns.violinplot(x='HeartDisease', y='Age', data=df)
sns.heatmap(df.corr(numeric_only=True), annot=True
```

7. DATA PRE-PROCESSING & CLEANING

In [7]:

```
df_encoded = pd.get_dummies(df, drop_first=True)
df_encoded
df_encoded = df_encoded.astype(int)
df_encoded
df_encoded.columns
```

8. IMPORT LIBRARIES

In[8]:

```
from sklearn.model_selection import train_test_split
# Import test train split to divide dataset into training
  and testing

from sklearn.preprocessing import StandardScaler
# Import StandardScaler to standardize feature values

from sklearn.metrics import accuracy_score, f1_score,
classification_report
# Import evaluation metrics to measure model
  performance

from sklearn.linear_model import LogisticRegression
# Import Logistic Regression classification algorithm

from sklearn.naive_bayes import GaussianNB
# Import Gaussian Naive Bayes classification
  algorithms

from sklearn.tree import DecisionTreeClassifier
# Import Decision Tree classification Algorithm

from sklearn.svm import SVC
# Important support Vector Machine(SVM) classification
  algorithm

from sklearn.neighbors import KNeighborsClassifier

# Import K-NN
```

9. FEATURE & TARGET SELECTION

In [9]:

```
X = df_encoded.drop('HeartDisease', axis = 1)
# select all columns except the target column as
  input features
y = df_encoded['HeartDisease']
# select the target column for prediction
```

10. TRAIN-TEST SPLIT

In [10]:

```
X_train, X_test, y_train, y_test = train_test_split(
X, y, stratify = y, test_size = 0.2, random_state = 42)
```

11. FEATURE SCALING

In [11]:

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

12. INITIALIZE ML MODELS

In [5]:

```
models = {
    "Logistic Regression": LogisticRegression(),
    "KNN" : kNeighborsclassifier(),
    "Naive Bayes": GaussianNB(),
    "Decision Tree" : DecisionTreeClassifier(),
    "SVM (RBF Kernal)": SVM (probability =True)
```

13. TRAIN & EVALUATE MODELS

In [6]:

```
results = [ ]
# create an empty list to store model evaluation results
for name, model in module.items( ):
# Train and evaluate each model one by one
    model.fit(X-train-scaled, y-train)
    # Train the model using training scaled data
    y-pred= model. predict (X-test - scaled)
    # Predict the target values for testing data
    acc = accuracy-score(y-test, y-pred)
    # calculate the accuracy of the model
    f1 = f1-score (y-test, y-pred)
    # calculate the F1 score of the model
    results.append({ 'Model':name,
                     'Accuracy': round(acc, 4),
                     'F1 score': round(f1, 4) })
    #store the model name & performance metrics in the
    result list
```

14. DISPLAY MODEL PERFORMANCE

In [14]:

```
results
```

15. SAVE BEST MODEL & PREPROCESSING FILES

In [15]:

```
import joblib
# Import Joblib to save trained models & objects
joblib.dump(models['KNN'], 'KNN_heart.pkl')
#save the trained KNN model to a file
joblib.dump(scaler, 'scaler.pkl')
#save the fitted standard scaler for future data
  preprocessing
joblib.dump(X.columns.tolist(), 'columns.pkl')
#save the feature column names used during
  training
```

16. VERIFY SAVED FILES

In [16]

```
['columns.pkl']
#Display the saved file name
```

Now we code on Vs code or use ai to get codes.

This code creates a streamlit web application for heart disease prediction. It loads the trained KNN model and scaler, collects patient details through an interactive interface, preprocesses the input data, predicts the risk of heart disease, & display the result as either High Risk or Low Risk.

```
import streamlit as st
#Import streamlit to build the web application
import pandas as pd
#Import Pandas for data handling
import joblib
#Import Joblib to load saved model files

"""For Load saved Model, scaler & expected columns"""
model = joblib.load("knn_heart_model.pkl")
scaler = joblib.load("heart_scaler.pkl")
expected_columns = joblib.load("Heart_columns.pkl")

"""collect user input"""
age = st.slider("Age", 18, 100, 40)
sex = st.selectbox("sex", ["M","F"])
chest_pain = st.selectbox("chest Pain Type", ["ATA","NAP",
             "TA","Asy"]
resting_bp = st.number_input("Resting Blood Pressure(
             mm Hg)", 800, 200, 120)
cholesterol = st.number_input("cholesterol(mg/dL)",
             100, 600, 200)
```

```
fasting_bs = st.selectbox("Fasting Blood Sugar >120
            mg/dL", [0,1])
resting_ecg = st.selectbox ("Resting ECG", ["Normal",
            "ST", "LVH"])
max_hr = st.slider("Max Heart Rate", 60, 220, 150)
exercise_angina = st.selectbox ("Exercise-Induced
        Angina", ["Y","N"])
Oldpeak = st.slider ("oldpeak (ST Depression)",0.0, 6.0,
            1.0)
St_slope = st.selectbox("ST slope", ["UP","Flat",
            "Down"])
"""When Predict is clicked"""
if st.button("Predict"):

    # create a raw input dictionary
    raw_input = {
        'Age' : age,
        'RestingBP' :resting_bp,
        'FastingBs': fasting_bs,
        'MaxHR' : max_hr,
        'Oldpeak' : oldpeak,
        'sex_' + sex : 1,
        'ChestPainType_' + chest_pain : 1,
        'RestingECG_' + resting_ecg : 1,
        'ExerciseAngina_' + exercise_angina : 1,
        'ST_slope_' + st_slope : 1
        }
```

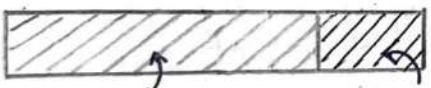
```
input_df = input_df [expected_columns]
# Reorder columns

scaled_input = scaler.transform (input_df)
# scaled the input
Prediction = model.predict (scaled_input) [0]
# Make prediction

"""Show Result"""
if prediction == 1 :
    st.error (" High Risk of Heart Disease")
else :
    st.success (" Low Risk of Heart Disease")
```

PART-4
(MODEL TUNING, ENSEMBLE & UNSUPERVISED LEARNING)

→ **Model Tuning** (Hyperparameter tuning): A process of finding the best hyperparameter values (like K=1,2,5,10 etc) to improve ML model's performance.

Since random hyper-parameters can lead to underfitting, overfitting or poor performance, tuning helps find the best combination so the model achieves better accuracy & generalizes well to unseen data.

→ **Cross validation**: It is a technique used to evaluate a model more reliably.

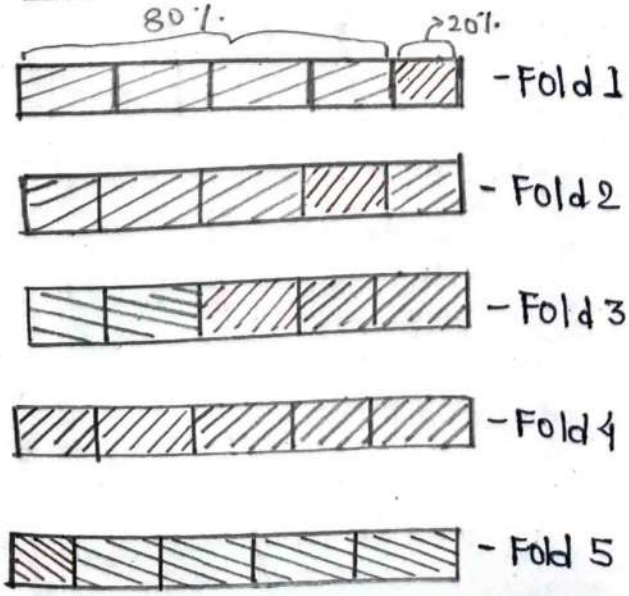


Instead of testing the model on a single train-test split, the dataset is divided into K folds.

The model is trained & tested in K times, with each K fold acting as the test set once.

The final performance is the average of all K accuracy scores.

Split in K = 5 Folds



- Each part use once as validation & remaining 4 parts for training

How K-Fold cross validation works

- Divide the dataset into K equal folds
- Use one fold for testing and the remaining folds for training.
- Repeat the process until every fold has been used as the test set once.
- Calculate the average accuracy of all K runs.

PROJECT: 1
(INSURANCE PROJECT)

In this "Insurance project" we use K-Fold cross validation is used in the project to validate the insurance prediction model & ensure reliable & consistent performance across different data splits.

→ cross validation Implementation

1. FEATURE & TARGET VARIABLE SELECTION

In [1]

```
X = df.drop('survived', axis=1)
y = df['survived']
```

2. FEATURE SCALING

In [2]

```
from sklearn.model_selection import cross_val_score
# Import function for performing K-Fold cross validation
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

3. K-FOLD CROSS VALIDATION

In[3]:

```
scores=cross_val_score(model_svm, X_scaled, y, cv=5,
        scoring ='accuracy')
#Perform 5 Fold Cross validation & calculate accuracy
 for each fold. before cross validation use that model &
 do all operations
print(scores) #Disply accuracy of each fold
print(scores.mean())
#Display the average cross-validation accuracy
```

→ **Hyperparameter tuning**: It is same as model tuning but this more technical specific name.

↳ **common hyperparameter**

- Ridge & Lasso Regression → (alpha (α) - Regularization parameter)
- K-Nearest Neighbors (KNN) → (n_neighbors = [3, 5, 7, 9])
- Decision-Tree → (max-depth, max_samples_split, max-features)
- Support Vector Machine (SVM) → (Kernel)
- Random Forest → (n_estimator, max-depth, max-features)

↳ **Hyperparameter Tuning Methods**

(1.) Manual Search

- Hyperparameter are selected manually.
- The model is trained repeatedly with different values
- Simple but time-consuming

(2.) GridSearchCV

(3.) RandomizedSearchCV

→ **GridsearchCV** : It is a hyperparameter tuning technique that automatically tests all possible combinations of hyperparameters using K-Fold cross validation and select. the combination with the highest average accuracy.

- **Grid**: A collection of all possible hyperparameter combinations to be tested
- **search**: Testing every combination of hyperparameter

↳ **How GridsearchCV works**

- create a grid of hyperparameter values.
- Test every possible combination.
- Perform K-Fold Cross Validation for each combination.
- Calculate the average accuracy of each combination
- select the combination with the best average performance.

↳ **example**

KNN - K-nearest Neighbors

N = [3, 5, 7, 9, 11, 13]

weights = ["uniform, "distance"]

metrics = ["manhatten", "eucledian"]

} These all parameter changes and test with each other

Now

N	weight	metric
3	uniform	manhatten
3	distance	manhatten
3	uniform	euledron
3	distance	euledian

→ experiment ⇒ Avg Acc

⇓

CV = 5.

PROJECT: 2
(IRIS HYPERPARAMETER TUNING)

This project uses GridsearchCV with 5-Fold Cross Validation to find the best SVM hyperparameter and improve classification performance on the Iris dataset.

↳ OpenGridCV Implementation

1. IMPORT LIBRARIES

In [1]:

```
import numpy as np
import pandas as pd
import seaborn as sns.
import matplotlib.pyplot as plt
```

2. LOAD AND EXPLORE DATASET

In [2]:

```
df = sns.load_dataset('iris')
df.head()
df['species'].unique()
# show unique target classes
```

3. PREPARE TRAINING & TESTING DATA

In [3]:

```
from sklearn.model_selection import train_test_split
X = df.drop ('species', axis=1)
y = df ['species']
X_train, X_test, y_train, y_test = train_test_split (
X, y, test_size = 0.33, random_state = 42)
```

4. TRAIN & EVALUATE KNN MODEL

In [4]:

```
from sklearn.neighbors import KNeighborsClassifier
model_knn = KNeighborsClassifier (n_neighbors = 13)
model_knn.fit (X_train, y_train)
model_knn.score (X_test, y_test)
```

5. TRAIN & EVALUATE SVM MODEL

In [5]:

```
from sklearn.svm import SVC
model_svm = SVC (gamma = 'auto')
model_svm.fit (X_train, y_train)
model_svm.score (X_test, y_test)
#Evaluate model accuracy
```

6. APPLY GRIDSEARCH CV

In [6]:

```
from sklearn.model_selection import GridsearchCV
# Import GridsearchCV

classifier = GridsearchCV(model_svm, {
#create Gridsearchcv object
            'c': [1,10,20,30],
            # Test Different C values
            'kernal': ['rbf','linear'],
            #Test different Kernals
            }                    ,cv=5,
                                  # 5 Folds
            return_train_score=False)
            #Do not return training scores

classifier.fit(X,y)
#Train Gridsearchcv
```

7. VIEW GRIDSEARCHCV RESULTS

In [7]:

```
classifier.cv_results_
#Display all over-validation results
results = pd.DataFrame(classifier.cv_results_)
#converts results into DataFrame
results
#Display results
```

→ RandomSearchCV

It is a hyperparameter tuning technique that randomly tests a fixed number of hyperparameter combinations using K-Fold cross validation. It is faster than Gridsearchcv & is suitable for large datasets & complex models.

↳ Working of Randomsearchcv

- **Random**: Randomly selects a fixed no. of hyperparameter combinations.
- **search**: Tests only the selected combinations.
- **CV (Cross Validation)**: Evaluates each combination using K-Fold cross validation
- **Best selection**: chooses the combinations with the highest average accuracy.

PROJECT: 2
(IRIS HYPERPARAMETER TUNING)

Here we are using RandomsearchCV to find the best hyperparameters for improved model performance.

8. APPLYING RANDOM SEARCH CV

In [8]:

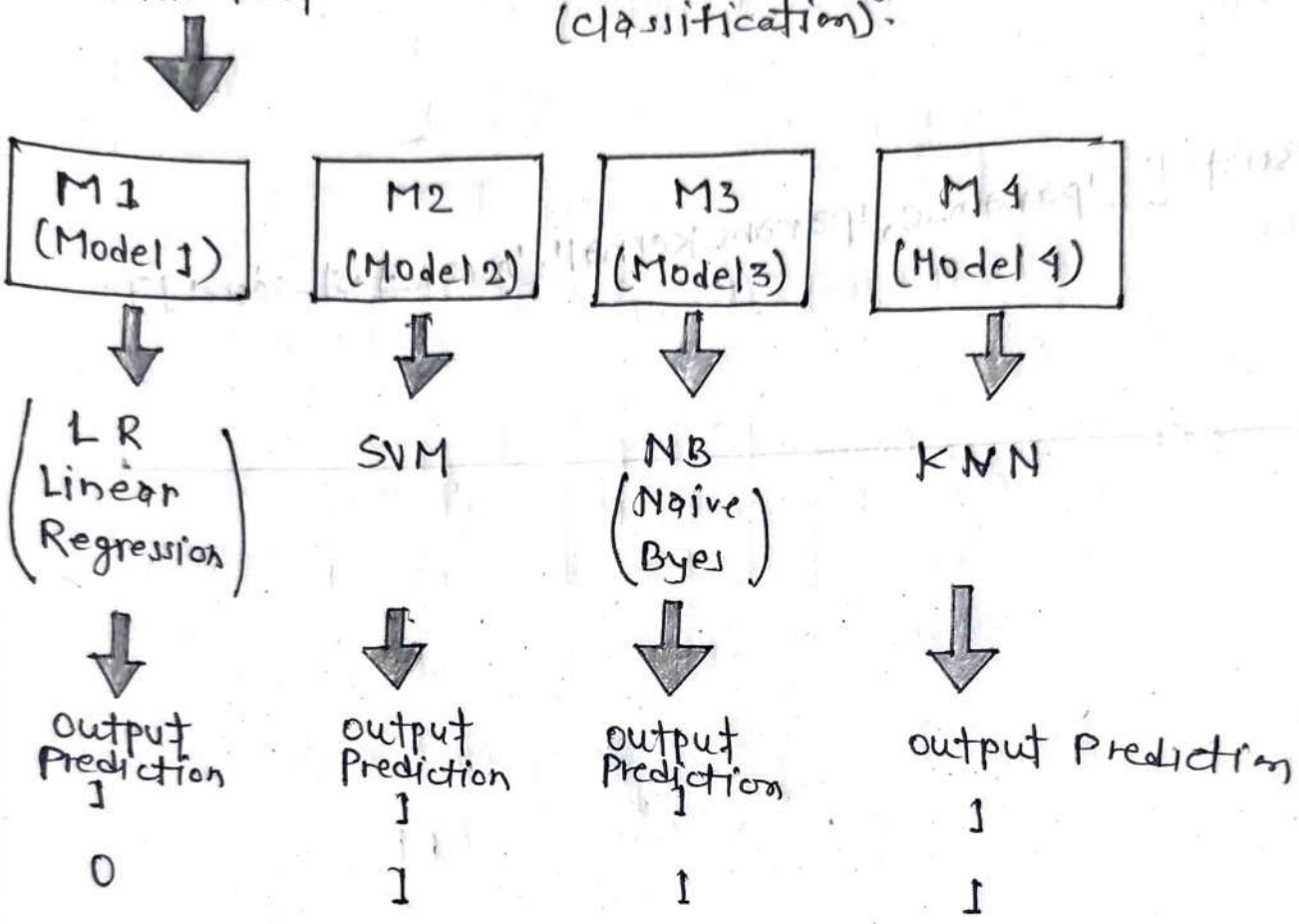
```
from sklearn.model_selection import
RandomizedSearchCV

classifier_r = RandomizedSearchCV(model_svm, {
#Create RandomizedsearchCV object
                'C': [1, 10, 20, 30],
                #Test Different C values
                'Kernal': ['rbf', 'linear'], }
                #Test Different kernals
                , n_ither=4,
                # Randomly test 4 parameter combination
                cv=5,
                #Used 5-fold cross validation
                return_train_score = False)
                #Do not return traing scores

classifier_r.fit(X, y)
# Train Randomizedsearchcv
```

9. VIEW RANDOMSEARCHCV RESULTS

In [9]:

```
results = pd.DataFrame(classifier_r.cv_results_)
#convert cv results into a DataFrame
results
#Display all search results
results[['param_C','param_kernal','mean_test_score']]
#Display selected result columns
```

→ **Ensemble Learning**: It combines multiple machine learning models to produce a more accurate & robust prediction than a single model. Generate the final prediction using majority voting (classification).



We hear the crowd
Mean SVM, NB, KNN all o/p is 1, 1 then the answer is 1, 1.

↳ **Working of Ensemble learning**

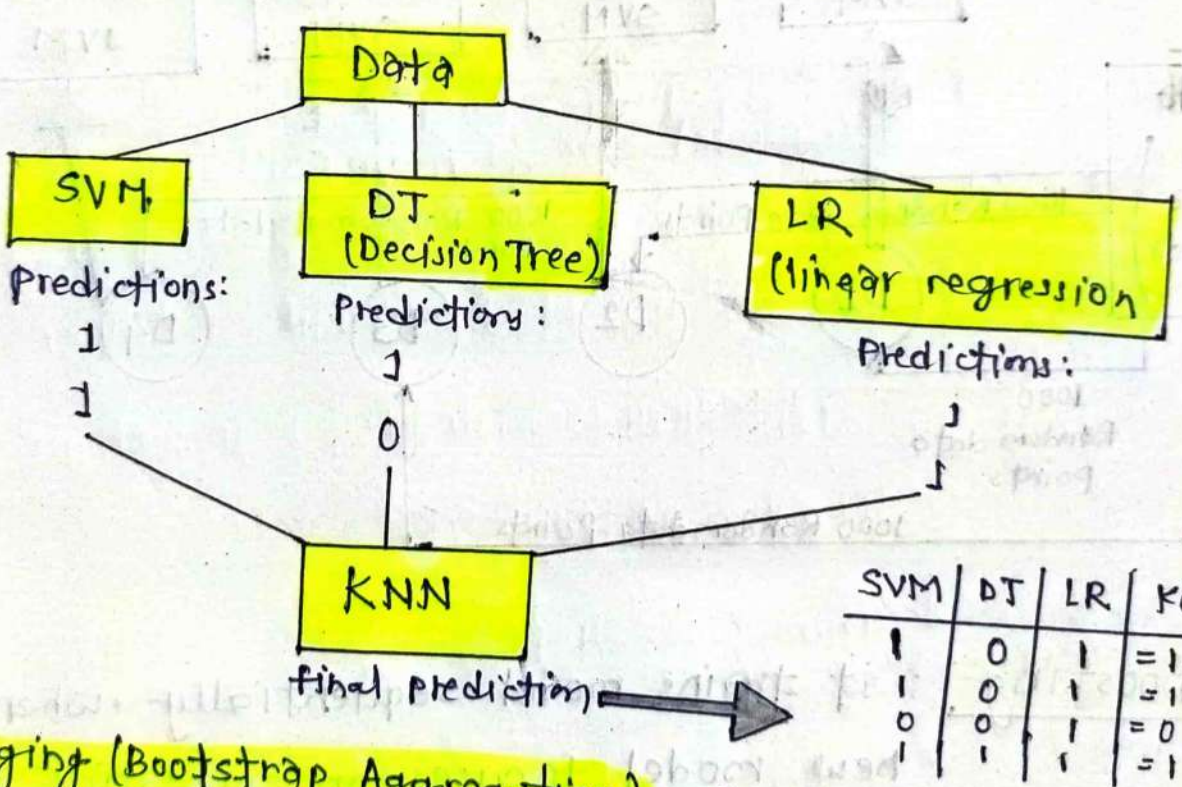
- Train Multiple ML models.
- Each model makes own prediction.
- Combine all prediction.
- Generate the final prediction using majority voting (classification) or averaging (regression)

→ # Types of Ensemble Learning

(1.) Stacking : It combines multiple base models & uses their predictions as input to a Meta Model (Final Decision Model) which learns how to make the final prediction.

↳ **working**

- Train Multiple base models.
- Collect their predictions.
- Train a Meta Model on these predictions
- Generate the final prediction.



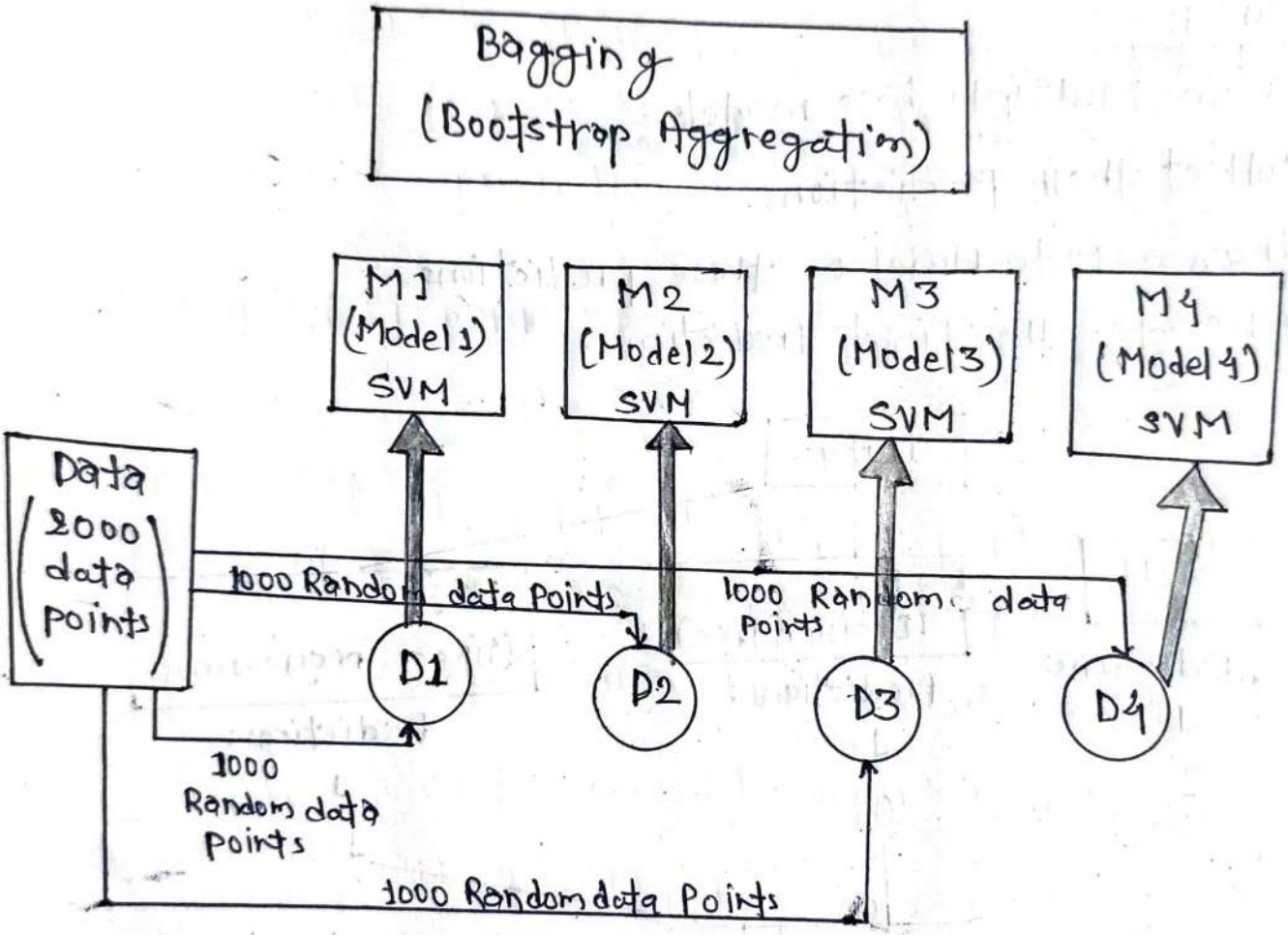
SVM	DT	LR	KNN
1	0	1	= 1
1	0	1	= 1
0	0	1	= 0
1	1	1	= 1

(2.) Bagging (Bootstrap Aggregation) : trains Multiple models of same type of different random subsets of the data & combines their predictions to reduce overfitting.

ex - RandomForest

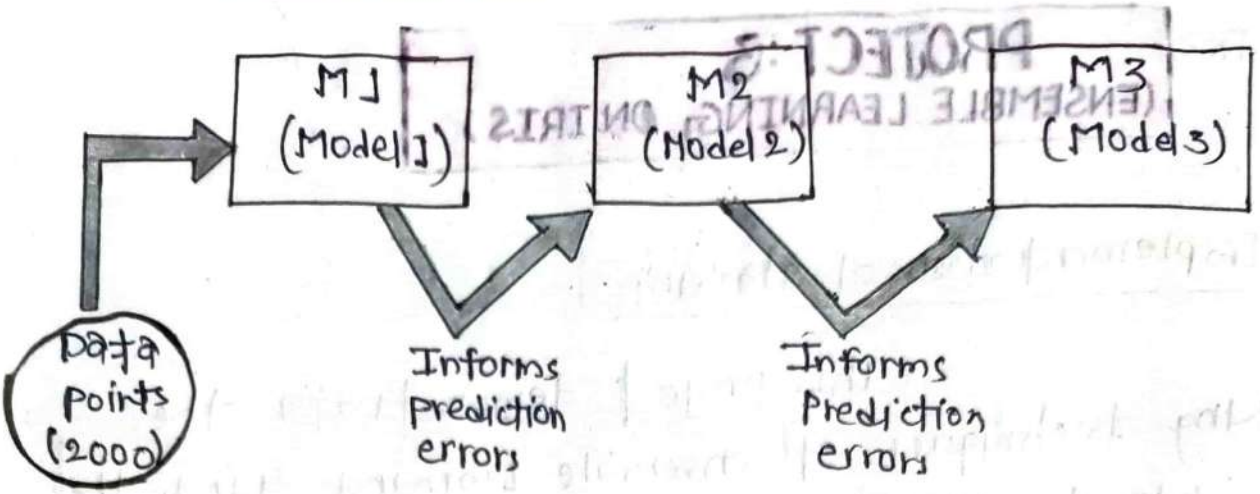
↳ **Working of Bagging**

- Create random data subsets (Bootstrapping)
- Train the same model on each subset
- Combine predictions using Majority voting or averaging



→ **Boosting** : It trains model sequentially, where each new model focuses on correcting the mistakes of previous model to improve overall performance.

Ex- XGBoost, AdaBoost, Gradient boosting.



↳ **Working**

- Train the first Model.
- Identify prediction errors.
- Train the next model to correct those errors.
- Repeat until the final strong model is Built.

PROJECT: 3
(ENSEMBLE LEARNING ON IRIS)

↳ code Implementation of stacking

This project demonstrate the stacking technique of ensemble learning using the Iris dataset. Multiple base model are combined with a meta-learner to Improve classification performance & evaluate prediction accuracy.

1. IMPORT LIBRARIES

In [1]:

```
from sklearn.model_selection import train_test_split
from sklearn.ensemble import StackingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score,
classification_report
import seaborn as sns
from sklearn.preprocessing import LabelEncoder
```

2. LOAD DATASET

In [2]:

```
df = sns.load_dataset('iris')
df.head()
```

3. SEPRATE FEATURES & TARGET

In [3]:

```
X = df.drop('species', axis=1)
# Remove target columns & store features
y = df['species']
# store target columns
```

4. ENCODE (LABEL)

In [4]:

```
le = LabelEncoder()
# create LabelEncoder object
y_encoded = le.fit_transform(y)
# convert text labels into numeric labels
```

5. SPLIT DATASET

In [5]:

```
X_train, X_test, y_train, y_test = train_test_split(X,
y_encoded, test_size = 0.2, random_state = 42,
stratify = y_encoded)
```

6. BASE LEARNERS

In [6]:

```
base_learners = [
    ('dt', DecisionTreeClassifier(random_state=42)),
    #Base Model 1: Decision Tree
    ('svc', SVC(probability=True, kernal='rbf', random_
    state=42)),
    #Base Model 2: SVM
    ('lr', LogisticRegression(max_iter=1000))
    #Base Model 3 : Logistic Regression
    ]
```

7. META LEARNER

In [7]

```
meta_learner = LogisticRegression(max_iter=1000)
```

8. CREATE STACKING MODEL & TRAIN

In [8]:

```
stacking_clf = StackingClassifier(estimators = base_learners,
                                  #Base learners model
               final_estimator = meta_learner, cv=5
               # Meta learner & 5 fold cross validation
stacking_clf.fit(X_train, y_train)
```

9. PREDICT & EVALUATE

In [9]:

```
y_pred = stacking_clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

↳ code Implementation of bagging

10. CREATE RANDOM FOREST MODEL

In [10]:

```
from sklearn.ensemble import RandomForestClassifier
rf_model = RandomForestClassifier(
        n_estimators = 100,
        # No. of Decision Trees
        max_depth = None,
        # Allow trees to grow fully
        random_state = 42, )
        # Ensure
```

11. TRAIN MODEL, PREDICT & EVALUATE

In [11]:

```
rf_model.fit(X_train, y_train)
y_pred = rf_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

↳ Implementation of Boosting, adaboost, gradient boost, Xgboost

12. IMPORT LIBRARIES

In [12]:

```
from sklearn.ensemble import AdaBoostClassifier,
GradientBoostingClassifier
#Import Adaboost, Gradient boost, XgBoost
```

13. ADABOOST CLASSIFIER

In [13]:

```
ada_model = AdaBoostClassifier(
            n_estimator = 100,
            #No. of weak learners
            random_state = 42)
            #Ensure reproducible results
ada_model.fit(X_train, y_train)
#Train AdaBoost model
y_pred = ada_model.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
accuracy
```

14. GRADIENT BOOSTING CLASSIFIER

In [14]:

```
gb_model = GradientBoostingClassifier(
        n_estimator = 100,
        learning_rate = 0.1,
        random_state = 42)
gb_model.fit (X_train, y_train)
y_pred = gb_model.predict (X_test)
accuracy = accuracy_score (y_test, y_pred)
accuracy
```

15. XGBOOST CLASSIFIER

In [15]:

```
xgb_model = XGBClassifier (
        n_estimator = 100,
        learning_rate = 0.1,
        max_depth = 3 # Max depth of each tree
        use_label_encoder = False,
        #Disable old label Encoder
        eval_metric = 'mlogloss',
        #Evaluation metric for multiclass classification
        random_state = 42
xgb_model.fit (X_train, y_train)
y_pred = xgb_model.predict (X_test )
accuracy = accuracy_score (y_test, y_pred)
accuracy
```

UNSUPERVISED LEARNING

It is type of ML where the training data does not contain label (target/ output values)

The model receives only input features and must discover hidden patterns, structures or relationships without knowing the correct answers.

↳ Supervised Learning Vs Unsupervised learning

Supervised Learning	Unsupervised learning
• It has input (X) & output (Y)	• only input data (X)
• Learns mapping from $X \rightarrow Y$	• No labels
• Used for classification & Regression.	• Learns patterns & hidden structures
	• Used for clustering, Dimensionality Reduction, & Anomaly Detection

↳ Goals of Unsupervised learning

- Group similiar data points (clustering)
- Reduce dimensions (PCA, t-SNE, UMAP)
- Detect anomalies/outliers.
- Discover hidden relationships
- Learn data distribution

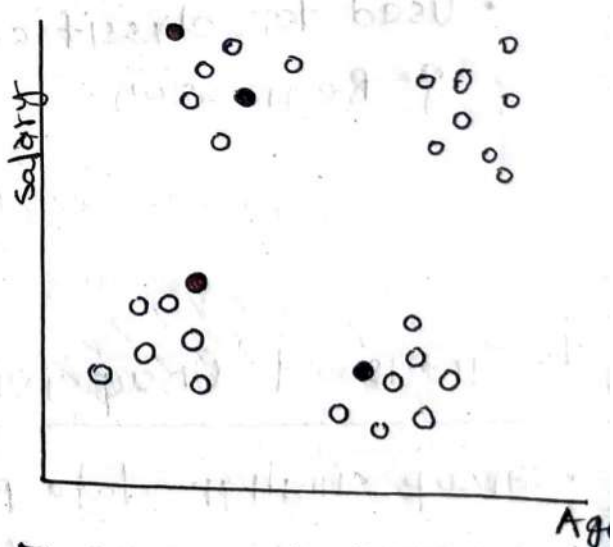
↳ common Applications

- customer segmentation
- Recommendation systems
- Fraud detection
- Image Compression
- Medical data Analysis
- Market Basket Analysis
- Network Intrusion Detection.

→ K-Means clustering

It is centroid-based unsupervised machine learning algorithm used to group similar data points into K clusters. since the data has no labels, the algorithm discovers patterns by itself.

↳ steps (Algorithm):



(1) Decide n clusters, K=4

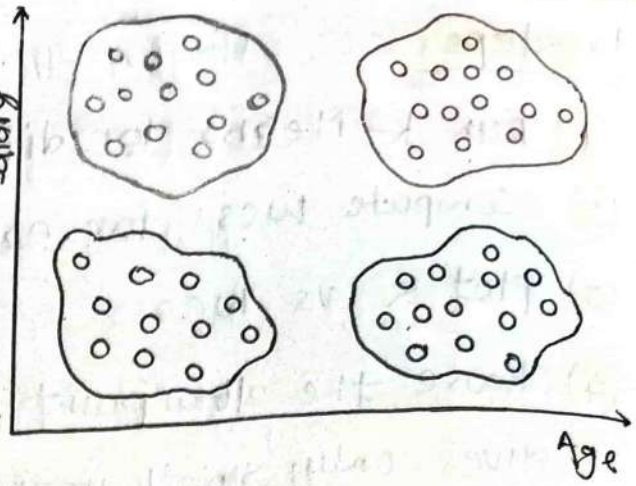
(2) Randomly initialize K centriods

(3) Assign clusters, Calculate the Euclidean distance from every data point to every centroid. Assign each point to its nearest centroid

(4) Re-assign the centerpoint : compute the new centroid as the mean of all points assigned to that cluster.

(5) Repeat steps 3 & 4 until:

- centroids stops changing,
  or
- Maximum iterations are reached.



*Euclidean Distance

For two points:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

The Nearest centroid Determines the cluster assignment.

* WCSS (within cluster sum of squares)

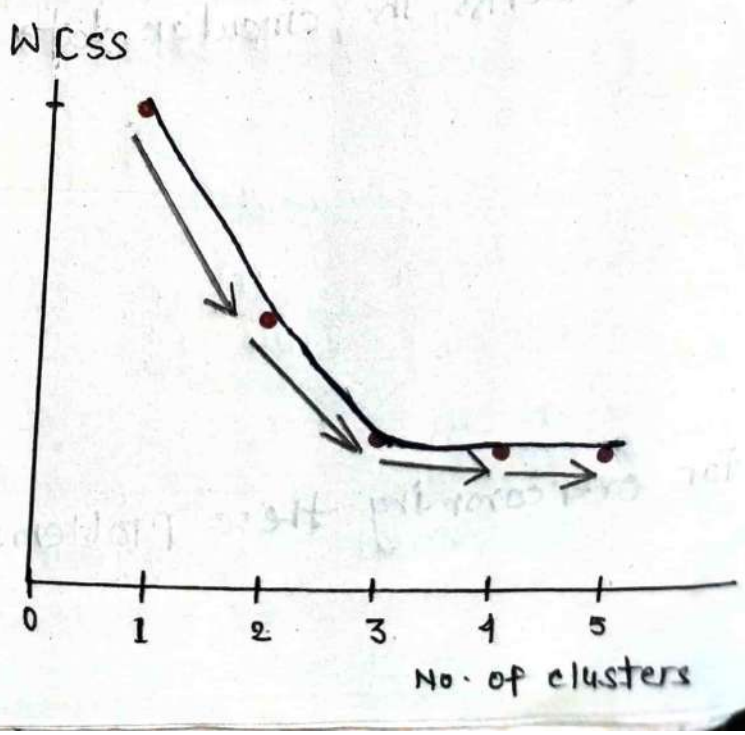
Measures how compact the clusters are.

$$WCSS = \Sigma (\text{Distance from point to centroid})^2$$

lower WCSS means tighter clusters.

→ Elbow Method

It is a technique used to determine the optimal no. of clusters (K) in the K-Means clustering algorithm.



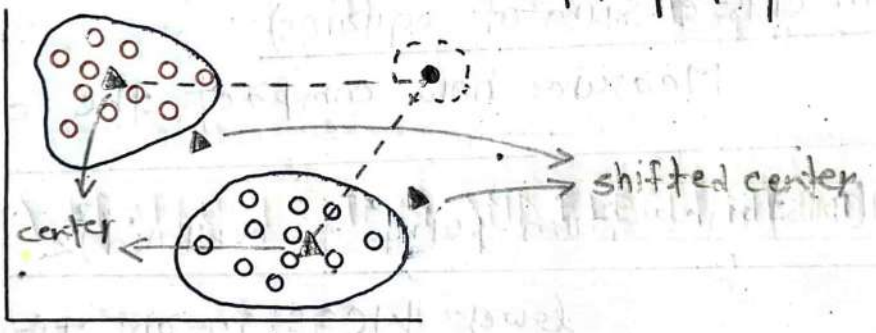
↳ steps

(1) Run K-Means for different values of K
(2) Compute WCSS for each K.
(3) Plot K vs WCSS.
(4) Choose the elbow point, where adding more clusters gives only small improvement

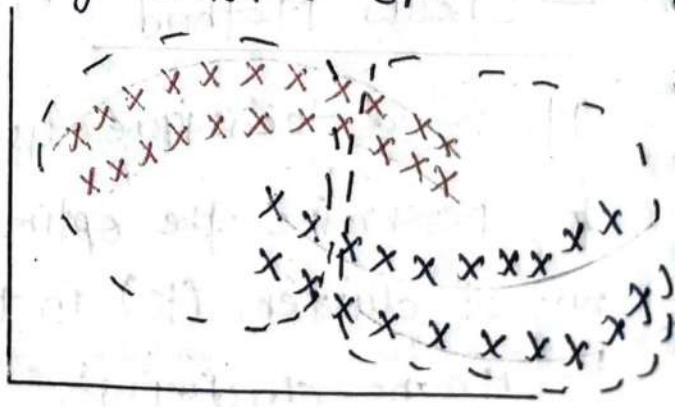
→ Disadvantages of K-Mean clustering

(1) We have intialize the K value, mean-Must choose K Manually

(2) outliers will move the center point



(3) Non circular data, it make wrong clusters or not works in circular data



For overcomming these problems we use "DBSCAN")

PROJECT: 4
(K MEANS CLUSTERING ON SYNTHETIC DATA)

↳ Overview

This project uses the K Means and DBSCAN algorithm to cluster a synthectic dataset. It uses the elbow Method to find the optimal number of clusters for K-Means and compares both algorithm on linear & non-linear data using scatter plot visualization.

→ Implementation of K-Means clustering algorithm

1. IMPORT LIBRARIES

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs
# Import function to generate synthectic clustered data
from sklearn.cluster import KMeans
# Import K-Means clustering algorithm
from sklearn.preprocessing import StandardScaler
# Import standardscaler to normalize features
```

2. GENERATE DATASET

In [2]:

```
X, y_true = make_blobs(  # Generate dataset
    n_samples = 500,
    # 500 samples
    centers = 3,
    # 3 clusters
    cluster_std = 4,
    # cluster spread = 4
    random_state = 42)

df = pd.DataFrame(X,
    columns = ['Feature_1', 'Feature_2'])
# Convert Numpy array into DataFrame
df
```

3. DATA PREPROCESSING AND ELBOW METHOD

In [3]:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
# standardize the feature

interia = []
# store interia values

K_range = range(1,11)
# Try K range values 1 to 10

for K in K_range:
    Kmeans = KMeans(n_cluster = K, random_state = 42)
    kmeans.fit(X_scaled)
    inertia.append(kmeans.inertia_)
# Train KMeans for each * & store interia
```

```
inertia
# Display inertia values
plt.plot(K_range, inertia, marker='o')
# plot Elow Method graph
```

3. TRAIN FINAL MODEL & VISUALIZE CLUSTERS

In [3]

```
kmeans_final = KMeans(n_clusters=3, random_state=42)
# create final kMeans model with optimal K=3
cluster_labels = kmeans_final.fit_predict(X_scaled)
# Train Model & Predict cluster labels
df['cluster'] = cluster_labels
# Add cluster labels to Dataframe
sns.scatterplot(
    x = df['Feature_1'],
    y = df['Feature_2'],
    hue = df['cluster'],
    Palette = 'viridis')
# Visualize clustered data
```

→ **DB-SCAN** (Density-Based spatial clustering of Applications with Noise)

It is a density-based unsupervised ML algorithm used to group data points based on density (how closely points are packed together) rather than using centroids like K-Means.

↳ **steps**

(1) choose value for:
- ϵ (epsilon)
- min_samples

(2) Randomly select a data point.

(3) Draw a circle of radius ϵ around the point.

(4) count the no. of neighboring points.

(5) Expand the cluster.

(6) stop when no more neighboring point can be added.

(7) Pick another unvisited point & repeat until all points are processed.

↳ **ex-** Imagine people standing in a park
- crowded groups become clusters.
- A person standing alone is marked a noise.

This is exactly how DBSCAN identifies dense region.

→

	K-Means	DBSCAN
Algorithm Type	centroid-based	Density-based
Need K	yes	No
Detect outlier	No	yes
Handles Noise	No	yes
cluster shape	circular/spherical	Any shape
Sensitive to outliers	yes	No
Uses centroids	yes	No

↳ Important parameters

(1) Epsilon (ϵ or eps) : Maxm distance within which two points are considered neighbors.

- Large ϵ → fewer, larger clusters.
- Small ϵ → more points become noise

(2) Min_samples : The Minm no. of neighboring points required to form a dense region

↳ Ex:-

eps = 0.3
min_samples = 5

A point must have atleast 5 neighbors within a radius of 0.3 to become part of a cluster.

Implementation of DBSCAN

4. IMPORT & GENERATE DATASET

In [4]:

```
from sklearn.datasets import make_moons
#Import function to generate moon-shaped dataset
X, y_true = make_moons (
    n_samples = 500,
    noise = 0.05,
    random_state = 42)
# Generate 500 moon-shaped samples with slight noise

from sklearn.cluster import KMeans, DBSCAN
#Import clustering algorithm
```

5. CREATE DATAFRAME & PREPROCESS DATA

In [5]:

```
df = pd.DataFrame (X, columns = ['Feature_1', Feature_2'])
# Convert dataset into a DataFrame
scaler = StandardScaler()
X_scaled = scaler.fit_transform (df)
```

6. APPLY K-MEANS CLUSTERING

In [6]

```
kmeans = KMeans (n_clusters = 2, random_state = 42)
# Create k-Means model with 2 clusters.
```

```
kmeans_labels = kmeans.fit_predict(X_scaled)
#Train Model & predict cluster labels
df['Kmeans_cluster'] = kmeans_labels
#store cluster labels
```

7. VISUALIZE K-MEANS RESULTS

In [9]:

```
sns.scatterplot(
    x= df['Feature_1'],
    y = df['Feature_2'],
    hue=df['kmeans_cluster'],
    palette = 'tab10')
#Plot K-Means clusters
```

8. APPLY & VISUALIZE DBSCAN

In [8]:

```
dbscan = DBSCAN(eps =0.3, minsamples = 5)
#Create DBSCAN Model
dbscan_labels = dbscan.fit_predict(X_scaled)
#Train model & predict cluster labels
df['dbscan_cluster'] = dbscan_labels
#store cluster labels
sns.scatterplot(
    x= df['Feature_1'],
    y = df['Feature_2'],
    hue = df['dbscan_cluster'],
    palette = 'tab10')
#Plot DBSCAN clusters
```

→ **Dimensionality Reduction** : It is the process of reducing the number of features in a dataset while preserving as much useful information as possible

↳ **Dimensionality** : Every input feature (column) in a dataset represents one dimension.

ex -
- $(x, y) \rightarrow 2D$
- $(x, y, z) \rightarrow 3D$
- 150 features → 150 Dimensional space

As the no. of features increased machine learning models start facing several problems.

↳ **curse of Dimensionality** : It refers to the problems that are arise when working with high-dimensional data.

• **Problems**

(1) **Difficult to Find patterns** : as dimensions increases, data becomes sparse & identifying meaningful patterns becomes harder

(2) **Distance-Based Algorithm Perform poorly**

In high dimensions, distance between points become less meaningful, reducing their performance.

(3) More storage Required : More features require more memory.

(4) Higher Training Time : A larger no. of features increases computational cost & model training time.

→ To achieve Dimensionality Reduction

(1) Feature selection

It removes unnessary features.

The original features remains unchanged; only irrelevant ones are discarded

Methods include:

- correlation
- Mutual Information
- chi-square Test
- Recursive Feature Elimination (RFE)

(2) Feature Extraction

Instead of removing features it combines existing feature into new ones.

Instead of two features, we now have one feature representing both.

PCA is the most popular Feature Extraction technique.

- Always some amount of Information will be lost but still we can predict.

→ Principal component Analysis (PCA)

It is an unsupervised dimensionality reduction technique used to reduce the no. of features in a dataset by transforming the original correlated features into a smaller set of new, uncorrelated feature called principal components (PCs) while preserving as much of the data variance (information) as possible.

↳ Used for:

- Reducing dimensionality
- Removes redundancy caused by correlated feature.
- speeds up model training
- Reduces storage requirements.
- Helps visualize high-dimens.
- Reduces the effect of the curse of dimensionality.

↳ steps

(1) collect the Dataset.
(2) standardize the Data
(3) Compute the covariance Matrix
(4) Compute Eigenvalues & Eigenvectors
(5) sort principal components
(6) select the Top Principal components
(7) Transform the data

PROJECT:5
(PCA FOR DIMENSIONALITY REDUCTION)

↳ **Overview:**

This project demonstrate Principal component Analysis (PCA) for dimensionality reduction. It reduces a 5-feature dataset to 2 Principal components while preserving maxm variance, making the data easier to visualize & analyze.

↳ **Implementation of PCA for dimensionality Reduction**

1. IMPORT LIBRARIES

In [1]:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import make_blobs
#generate a synthectic dataset
from sklearn.preprocessing import StandardScaler
#standardize feature
from sklearn.decomposition import PCA
# Import PCA
```

2. GENERATE DATASET & STANDARDIZE

In [2]:

```
X, y = make_blobs(
    n_samples = 500,
    n_features = 5,
    centers = 3,
    clusters_std = 1.5,
    random_state = 42)
# Generate 500 samples with 5 feature & 3 clusters
scaler = StandardScaler
X_scaled = scaler.fit_transform(X)
```

3. APPLY PCA & CREATE DATAFRAME

In [3]:

```
pca = PCA(n_components = 2)
# Reduces 5 features into 2 principal components
X_pca = pca.fit_transform(X_scaled)
df_pca = pd.DataFrame(X_pca, columns = ['PC1', 'PC2'])
# Convert PCA output into a DataFrame
df_pca['label'] = y
# Add original labels for visualization
```

4. VISUALIZE PCA RESULTS

In [4]:

```
sns.scatterplot( data = df_pca ,
                 x= 'PC1' ,
                 y = 'PC2',
                 hue = 'label' ,
                 palette = 'set2' )
# PC1 → first Principal component
    (capures maxm variance)
#PC2 → second Principal component
    (capures second highest variance)
plt.title("PCA Visualization")
plt.show()
```